

Highlights

A Stackelberg Inspection Game for Diagnosis–Repair in LLM-Based Empty Container Repositioning Optimization

Kangzhen Peng, Siqi Duan, Yuhong Wang, Jiaxin Cai, Shida Xu, Zhihong Jin

- Diagnosis–repair is cast as a Stackelberg inspection game defined on a LangGraph state machine as the stage-game apparatus.
- The inspector commits, first-mover, to an evidence-ranked probing policy with executed refutation of every repair by re-solve.
- Probing is bound to the committed order, so first-probe naming equals the ranking’s alignment; round-one confirmation is separately gated by the repair passing the re-solve.
- Committed inspection dominates single-judge, debate, and reflexion naming at the lowest token cost; guilty layers comply at first contact under commitment, and the rare deflections observed (all in non-committed arms) follow an overturned repair.

A Stackelberg Inspection Game for Diagnosis–Repair in LLM-Based Empty Container Repositioning Optimization

Kangzhen Peng^a, Siqi Duan^a, Yuhong Wang^b, Jiaxin Cai^c, Shida Xu^d and Zhihong Jin^{a,*}

^aCollege of Transportation Engineering, Dalian Maritime University, No. 1 Linghai Road, Dalian, 116026, Liaoning, China

^bFaculty of Maritime and Transportation, Ningbo University, No. 169, Qixing South Road, Ningbo, 315211, Ningbo, China

^cSchool of Economics and Management, Liaoning University of Technology, Jinzhou, 121001, Liaoning, China

^dCollege of Economics and Management, Dalian Jiaotong University, Dalian, 116028, Liaoning, China

ARTICLE INFO

Keywords:

large language model
multi-agent system
Stackelberg game
finite-state machine
automated optimization modeling
empty container repositioning

ABSTRACT

We recast the error diagnosis and repair stage of an LLM multi-agent pipeline for Empty Container Repositioning (ECR) optimization as a Stackelberg *inspection game* defined on a state machine. A LangGraph finite-state machine orchestrates role-specialized agents (Data Engineer → Model Expert → Python Developer → Solver) and serves as the stage-game apparatus. When a solve fails, the diagnosis agent acts as an *inspector* (the Stackelberg leader) that commits, first and observably, to an inspection policy: an evidence-ranked probing order together with *executed refutation* — verifying every response by a real re-solve rather than by cheap talk. The accused agents are *inspectees* (the followers): because a downstream layer is routinely implicated by an upstream defect it did not cause, each accused agent has a genuine private incentive to deflect blame, so its interests are not aligned with truthful attribution. We formalize the mechanism and state a proposition linking committed inspection to a reduced viable root-cause set, then test it on a *demand-uncertainty* sea–land ECR problem formulated as a two-stage stochastic linear program: three fault plants (one per layer), three problem instances, five diagnosis protocols, and shared frozen failure blackboards. The measured mechanism is precise: probing is bound to the committed order, so first-probe naming is exactly the ranking’s alignment — which holds on every model-layer board and fails only for rank-invisible semantic data faults — while on the main grid only 14 of the 30 aligned first probes are confirmed by the round-one re-solve (9 overturned repairs, 7 attributions stolen by downstream regeneration), and a misaligned probe can conversely absorb an upstream fault outright; causal commitment dominates single-judge, debate, and reflexion naming (paired across all thirty shared boards without a single discordance) at the lowest token cost, the guilty layer complies at first contact under commitment (the rare deflections observed all sit in non-committed arms and follow an overturned repair), and 43–71% of complying repairs are overturned by the re-solve — executed verification, not talk, carries attribution. The order advantage replicates conditionally on the ranking’s ability to see the fault layer; for rank-invisible semantic data faults we register a design revision rather than a post-hoc fix.

1. Introduction

Empty Container Repositioning (ECR) is a standing cost center in global liner shipping. Trade imbalance leaves empty equipment stranded at import hubs while export regions face shortages; planners must move empties across sea and inland networks under capacity, transit-time, and inventory limits (Poiani et al., 2024; Wang et al., 2025a). Under demand uncertainty the problem is naturally a multi-stage (often two-stage) stochastic program: first-stage repositioning commitments are made before scenario-dependent export demand realizes, and recourse actions (e.g. leasing or spill) absorb residual imbalance (Poiani et al., 2024; Cai et al., 2025). Building a correct model therefore requires scarce domain expertise: admissible sea–dry topologies, mode-specific costs, scenario trees or deterministic equivalents, and recourse design that remains relatively complete. Small misspecifications in data, mathematics, or solver code routinely yield infeasible, suboptimal, or operationally meaningless plans.

*Corresponding author.

✉ pengkangzhen@dlmu.edu.cn (K. Peng); duansiqi@dlmu.edu.cn (S. Duan); wangyuhong@nbu.edu.cn (Y. Wang); jgcailx@lnut.edu.cn (J. Cai); xushida@djtu.edu.cn (S. Xu); jinzhijhong@dlmu.edu.cn (Z. Jin)

🌐 <https://orcid.org/0000-0003-4342-1561> (Z. Jin)

ORCID(s):

Traditional practice addresses ECR with expert-built mixed-integer or stochastic programs, meta-heuristics, and, more recently, multi-agent reinforcement learning for joint repositioning and fleet decisions (Poiani et al., 2024; Cai et al., 2025). These methods can be strong once a model exists, but they share a structural bottleneck: every new network, planning window, or uncertainty description must be *hand-modeled* by OR specialists. Reformulation is slow; knowledge is hard to transfer across carriers; and the gap between how operations are described in language and how they appear in an LP remains human-mediated. Learning-based controllers alleviate online decision load but still presume a correctly engineered state, action, and reward interface. In short, the limiting resource is often not the solver, but the modeling labor that feeds it.

Large language models (LLMs) change that interface. They parse operational narratives and structured tables, propose mathematical formulations, emit executable solver code, and iterate against tool feedback (Xiao et al., 2024; AhmadiTeshnizi et al., 2024a). These abilities align with ECR pain points: the input is already a mixture of natural language and instance data; the desired output is a compilable, solvable model whose quality can be checked by an industrial MIP solver and, when available, by a known optimum z^* . Multi-agent scaffolds (role-specialized data, modeling, and coding agents coordinated by a shared state) further match the layered work of OR practice, from data preparation through formulation to implementation (Wu et al., 2024; Xiao et al., 2024; AhmadiTeshnizi et al., 2024a). Recent maritime applications already use LLM agents for stowage-related planning and sketch agent-based transportation modeling (Liu et al., 2025), indicating that the same automation path is opening for container logistics.

Automation, however, relocates rather than removes the reliability problem. In a natural-language optimization (NLO) pipeline, defects propagate along *data* $\triangleright$ *model* $\triangleright$ *code*: an upstream modeling error often surfaces as a downstream crash or wrong objective, so stack traces mark the symptom locus rather than the certified root cause. Without domain knowledge the pipeline cannot formulate ECR correctly in the first place; with knowledge, failed solves still demand *attribution and repair*. Existing agentic OR systems rely mainly on single-judge accusation, sequential backward repair, multi-agent debate, or verbal reflection (Du et al., 2023; Shinn et al., 2023; Liang et al., 2024). These protocols increase deliberation but treat diagnosis as (multi-call) classification with cheap-talk self-checks. They do not commit an observable inspection policy, do not verify accused replies by executed re-solve as a discipline, and do not account for the private incentive of an innocent downstream layer to deflect blame.

Thesis. We argue that diagnosis–repair under layered ECR modeling errors is a Stackelberg (von Stackelberg, 1952) *inspection game* (Avenhaus et al., 2002; Maschler, 1966). A diagnosis agent acts as inspector (leader) and commits, first and observably, to a policy $\sigma = (\omega, \nu)$: a probing order ω aligned with causal dependence, and a verification rule ν of *executed refutation* (a real re-solve under a strict-success test against z^*). Accused data, model, and code agents are inspectees (followers) that choose to comply-and-repair or to deflect. Commitment gives the mechanism teeth: a guilty layer’s best response is to repair; an innocent layer’s deflection is upheld only if re-solve confirms it. We instantiate the game on a LangGraph state machine as stage-game apparatus and validate it on a demand-uncertainty sea–land ECR two-stage DEP (Section 3).

Contributions. (i) We cast diagnosis–repair in an LLM multi-agent NLO pipeline as a Stackelberg inspection game on a finite-state machine. (ii) We specify an inspector commitment to causal-order probing with executed refutation, inducing verifiable attribution rather than confidence-routed accusation. (iii) We state a proposition that, under explicit premises on causal layering and repair-path behaviour, causal-order inspection strictly reduces the viable root-cause set relative to single-judge play without dropping the true layer, and test it with commitment-order ablations and baselines on demand-uncertainty ECR. (iv) We separate progressive domain-knowledge injection (necessary for ECR solvability) from the game-theoretic diagnosis claim (necessary for auditable repair).

Section 2 reviews ECR research and the shift to agentic NLO. Section 3 gives the ECR problem description and mathematical formulation. Section 4 develops the inspection mechanism. Section 5 reports the experimental campaign and results; Section 6 concludes.

2. Related Work

2.1. Empty container repositioning

ECR has long been studied as a network optimization problem driven by directional trade imbalance. Classical and contemporary OR formulations use deterministic or stochastic mathematical programs for repositioning, inventory, slot allocation, and related fleet decisions, often solved by exact methods or meta-heuristics (Wang et al., 2025a; Cai

et al., 2025). Under uncertainty and operational black-box constraints, configurable partially observable models and two-stage learning procedures have been proposed for joint repositioning and fleet deployment (Poiani et al., 2024). These lines establish ECR as a hard, domain-rich benchmark with verifiable optima once a model is fixed. They do not automate the passage from an operational description to that model. Emerging LLM-agent work in maritime and transportation systems (e.g. text-to-stowage pipelines and conceptual LLM-agent transportation frameworks) (Liu et al., 2025) suggests that natural-language interfaces are becoming plausible for container logistics, but reliability of the modeling–coding–solving chain remains open. We therefore use demand-uncertainty ECR as a *validation bed* with an expert DEP optimum z^* , not as a claim of a new ECR theory.

2.2. From expert modeling to agentic natural-language optimization

The broader OR+LLM literature is shifting how optimization problems are produced and checked. Chain-of-Experts orchestrates role-specialized agents with forward construction and backward reflection for complex OR tasks (Xiao et al., 2024); OptiMUS modularizes formulation, coding, and evaluation against solver feedback at scale (AhmadiTeshnizi et al., 2024a,b); related expert-guided reasoning pipelines further target automated optimization modeling (Yang et al., 2025). Relative to classical “expert writes the math, solver optimizes,” this body of work exhibits three coupled shifts. First, the *object of study* moves from solving a given mathematical program to turning unstructured descriptions and data into a solvable, correct model. Second, *verification* moves from textual resemblance or manual reading toward executable checks (compile, solve, and comparison to z^* when available), so success is judged by whether the artifact passes a solver rather than by whether the prose looks right. Third, once modeling is a multi-stage pipeline, *reliability* becomes first-class: errors cascade along data, model, and code; the failure surface is often downstream of the true root cause. Domain knowledge injection is necessary for problems such as ECR to be formulable at all; failure attribution and diagnosis–repair are necessary for the pipeline to recover when it does not. Our work sits in this third shift while retaining executable solver verification as the hard truth signal.

2.3. Reliability mechanisms in LLM multi-agent systems

Layered LLM multi-agent systems face a distinctive reliability problem: errors cascade across role boundaries, and the observable failure surface (crash, infeasibility, or wrong objective) is often downstream of the true root cause. In NLO pipelines this appears as data $\triangleright$ model $\triangleright$ code dependence—a modeling mistake may surface as a coding crash or a wrong solve. Domain knowledge and role scaffolding are a *prerequisite* for hard OR tasks to be formulable at all (Wu et al., 2024; Xiao et al., 2024; AhmadiTeshnizi et al., 2024a), but they do not by themselves certify which layer failed or compel a correct repair. A complementary line therefore treats verification as executable rather than textual: compile, solve, and check against solver feedback or a known optimum when available (AhmadiTeshnizi et al., 2024a,b). Two further mechanism families address post-failure recovery. Failure attribution asks which agent and which step caused the failure: Who&When shows that even strong LLM judges remain far from reliable step-level attribution (Zhang et al., 2025); subsequent work stresses full-trace observability (Chen et al., 2026), hierarchical or causal-graph attribution (Wang et al., 2026), and Causal Agent Replay that intervenes on trajectory steps and re-executes forward (Shah, 2026). Diagnosis–repair and multi-call protocols then try to act on that judgment: single-judge accusation with status-based priors, reverse-order sequential repair, multi-agent debate (Du et al., 2023; Liang et al., 2024), and Reflexion-style verbal memory (Shinn et al., 2023). These lines increase deliberation or observability, yet they typically stop at naming (or reflective re-prompting). Missing for layered OR pipelines is a *committed* inspection policy whose replies are settled by re-solve under a strict success rule—the incentive-compatible, executed-refutation mechanism we develop as a Stackelberg inspection game in Section 2.4.

2.4. Game-theoretic multi-agent LLM systems

Game-theoretic views of LLM multi-agent systems cover debate, Nash-style coordination, and Stackelberg resource governance (Yi et al., 2025; Wang, 2026; Du et al., 2023). Classical inspection games analyze an inspector who commits to a monitoring policy and an inspectee who best-responds (Avenhaus et al., 2002; Maschler, 1966). In AI, the same leader-commitment logic underpins deployed Stackelberg security games, where a defender commits to a randomized patrolling or scheduling policy and adversaries best-respond after surveillance (Korzhyk et al., 2011; Tambe, 2011). We use that template for layered NLO diagnosis–repair: the diagnosis agent is the inspector; data/model/code agents are inspectees; commitment is to causal-order probing plus executed refutation on the ECR blackboard. This differs from resource Stackelberg controllers (token or tool budgets) and from debate equilibria (message-passing agreement) in both players and payoffs. The empirical spine is whether committed causal order improves *verifiable* attribution relative to unordered commitment and to non-committed single-judge or multi-call baselines (Section 5).

3. Problem Description and Mathematical Formulation

3.1. Problem description

We validate the diagnosis–repair mechanism on a sea–land Empty Container Repositioning (ECR) problem under *export-demand uncertainty*. The application model is a two-stage stochastic linear program on a weekly planning window (deterministic equivalent / DEP), adapted from our companion formulation for demand-uncertainty ECR. Unlike the MAKO baseline’s deterministic shipper–consignee multi-commodity flow, the present benchmark retains only sea and dry ports, treats empty supply as exogenous, and places all randomness in scenario-dependent export demand. Agents in the pipeline must formulate and solve this DEP from a natural-language description plus structured instance data; the expert DEP optimum supplies the strict-success rule used by the diagnosis mechanism (Section 4.4).

The network is a directed multigraph on node set $\mathcal{N} = \mathcal{H} \sqcup \mathcal{L}$, where sea ports $\mathcal{H} = \mathcal{H}^{\text{hub}} \sqcup \mathcal{H}^{\text{spoke}}$ partition into global hubs and regional spokes, and $\mathcal{L}$ is the set of dry ports. Inland moves use modes $\kappa \in \mathcal{K}$ (road, rail). Rather than writing Big-M hinterland restrictions inside the LP, we *compile* topology into an admissible arc set

$$\mathcal{A} \subseteq \mathcal{N} \times \mathcal{N} \times \mathcal{K}, \quad (1)$$

built once from hinterland indicators λ_{ij} : include sea–dry arcs (i, j, κ) and (j, i, κ) iff $\lambda_{ij} = 1$, and dry–dry arcs (j_1, j_2, κ) iff j_1 and j_2 share a common hinterland sea port. Each arc $a = (i, j, \kappa) \in \mathcal{A}$ carries transit time τ_a , unit repositioning cost c_a^{repo} , and capacity F_a^{cap} . Flow variables exist only on $\mathcal{A}$; inadmissible hinterland pairs simply never appear.

Time is discrete by week. Empty containers held at sea or dry ports serve export demand. The empty supply $\xi_{n,t}$ at node n in week t is an *exogenous deterministic* parameter (no laden-to-empty conversion dynamics). The core uncertainty is the scenario-dependent export demand $\eta_{n,t}^\omega$ for $\omega \in \Omega$ with probabilities π_ω . Leasing is unbounded (at a premium); a heavily penalized spill variable ensures relatively complete recourse under storage caps.

A full operational system may wrap successive windows in a rolling horizon. For the agentic modeling task we fix a single window $\mathcal{T} = \{1, \dots, H\}$ (smoke instance: $H=4, |\Omega|=5$) and ask the pipeline to build the extensive-form DEP of the two-stage program below. All decisions are continuous nonnegative TEUs; at weekly scale the integrality gap is negligible.

3.2. Mathematical formulation

3.2.1. Sets, parameters, and variables

Sets. $\mathcal{H}^{\text{hub}}, \mathcal{H}^{\text{spoke}}, \mathcal{L}; \mathcal{N}^{\text{inv}} = \mathcal{H} \cup \mathcal{L}$; modes $\mathcal{K}$; admissible arcs $\mathcal{A}$ as in (1); scenarios Ω ; window $\mathcal{T}$. For each node $n \in \mathcal{N}$ define the forward and backward stars

$$\mathcal{A}_n^+ = \{(n, j, \kappa) \in \mathcal{A}\}, \quad \mathcal{A}_n^- = \{(i, n, \kappa) \in \mathcal{A}\}. \quad (2)$$

Flow variables are indexed on arcs $(i, j, \kappa) \in \mathcal{A}$; we write $x_{ijk,t}^\omega$ interchangeably with $x_{a,t}^\omega$ for $a = (i, j, \kappa)$.

Deterministic parameters. Exogenous supply $\xi_{n,t}$; spoke top-up $E_{i,t}$; initial inventory $I_{n,0}$; committed inland arrivals $\bar{Q}_{n,t}^{\text{arr}}$ (zero on the first-window smoke instance); hub vessel calls $V_{i,t} \in \{0, 1\}$ and per-call bounds $B_i^{\text{in}}, B_i^{\text{out}}$; effective external export floor $\tilde{D}_{i,t}^{\text{ext}} = \min\{D_{i,t}^{\text{ext}}, B_i^{\text{out}}\}$; per-arc $(\tau_{ijk}, c_{ijk}^{\text{repo}}, F_{ijk}^{\text{cap}})$ for $(i, j, \kappa) \in \mathcal{A}$; storage caps U_n^{cap} ; sea costs $c^{\text{sea,in}}, c^{\text{sea,out}}$; repositioning / holding / leasing / spill costs $c_{ijk}^{\text{repo}}, c_n^{\text{hold}}, c_n^{\text{lease}}, c^{\text{spill}}$, with $c^{\text{spill}} \gg \max\{c_n^{\text{lease}}, c_n^{\text{hold}}, c_{ijk}^{\text{repo}}\}$.

Random parameters. Demand $\eta_{n,t}^\omega$ and probabilities π_ω ($\sum_\omega \pi_\omega = 1$). Scenarios are drawn by truncated-normal additive shocks about nominal means $\bar{\eta}_{n,t}$.

First-stage decisions (here-and-now). $y_{i,t}^{\text{in}}, y_{i,t}^{\text{out}} \in \mathbb{R}_+$ for $i \in \mathcal{H}^{\text{hub}}, t \in \mathcal{T}$: hub sea arrivals (arrival-week accounting) and departures, chosen *before* demand is revealed.

Second-stage decisions (wait-and-see). For each $\omega \in \Omega$: arc flows $x_{ijk,t}^\omega$ ($(i, j, \kappa) \in \mathcal{A}$), inventories $I_{n,t}^\omega$, leases $r_{n,t}^\omega$, and spill $s_{n,t}^\omega$, all in $\mathbb{R}_+$.

3.2.2. Mathematical model

The inland submodel is a *node–arc* network formulation on the admissible digraph $(\mathcal{N}, \mathcal{A})$: topology is encoded by the stars (2), and conservation at each node is written with incidence sums over $\mathcal{A}_n^\pm$.

Stage 1 master problem [D1].

$$\min_{\mathbf{y}} \sum_{t \in \mathcal{T}} \sum_{i \in \mathcal{H}^{\text{hub}}} (c^{\text{sea}, \text{in}} y_{i,t}^{\text{in}} + c^{\text{sea}, \text{out}} y_{i,t}^{\text{out}}) + \sum_{\omega \in \Omega} \pi_{\omega} \Theta^{\text{D}}(\mathbf{y}, \omega) \quad (3)$$

$$\text{s.t. } y_{i,t}^{\text{in}} \leq B_i^{\text{in}} V_{i,t}, \quad \forall i \in \mathcal{H}^{\text{hub}}, t \in \mathcal{T}, \quad (4)$$

$$y_{i,t}^{\text{out}} \leq B_i^{\text{out}} V_{i,t}, \quad \forall i \in \mathcal{H}^{\text{hub}}, t \in \mathcal{T}, \quad (5)$$

$$y_{i,t}^{\text{out}} \geq \tilde{D}_{i,t}^{\text{ext}} V_{i,t}, \quad \forall i \in \mathcal{H}^{\text{hub}}, t \in \mathcal{T}, \quad (6)$$

$$y_{i,t}^{\text{in}}, y_{i,t}^{\text{out}} \geq 0. \quad (7)$$

Stage 2 recourse [D2]. Given $\mathbf{y}$ and realized η^{ω} , write the node–arc inflow and outflow through the stars (2) as

$$\begin{aligned} \text{In}_{n,t}^{\omega} &= \sum_{(i,n,\kappa) \in \mathcal{A}_n^-} x_{in\kappa,t-\tau_{in\kappa}}^{\omega}, \\ \text{Out}_{n,t}^{\omega} &= \sum_{(n,j,\kappa) \in \mathcal{A}_n^+} x_{nj\kappa,t}^{\omega}, \end{aligned} \quad (8)$$

with the convention $x_{ijk,s}^{\omega} := 0$ if $s \notin \mathcal{T}$ (lagged arrivals outside the window enter via $\bar{Q}_{n,t}^{\text{arr}}$). The recourse problem is

$$\Theta^{\text{D}}(\mathbf{y}, \omega) = \min_{\mathbf{x}, \mathbf{I}, \mathbf{r}, \mathbf{s}} \left(C^{\text{repo}}(\omega) + C^{\text{hold}}(\omega) + C^{\text{lease}}(\omega) + C^{\text{spill}}(\omega) \right), \quad (9)$$

where the four activity cost blocks (repositioning / holding / leasing / spill) are

$$C^{\text{repo}}(\omega) = \sum_{t \in \mathcal{T}} \sum_{(i,j,\kappa) \in \mathcal{A}} c_{ijk}^{\text{repo}} x_{ijk,t}^{\omega}, \quad (10)$$

$$C^{\text{hold}}(\omega) = \sum_{t \in \mathcal{T}} \sum_{n \in \mathcal{N}^{\text{inv}}} c_n^{\text{hold}} I_{n,t}^{\omega}, \quad (11)$$

$$C^{\text{lease}}(\omega) = \sum_{t \in \mathcal{T}} \sum_{n \in \mathcal{N}^{\text{inv}}} c_n^{\text{lease}} r_{n,t}^{\omega}, \quad (12)$$

$$C^{\text{spill}}(\omega) = \sum_{t \in \mathcal{T}} \sum_{n \in \mathcal{N}^{\text{inv}}} c_n^{\text{spill}} s_{n,t}^{\omega}. \quad (13)$$

subject to

$$\begin{aligned} I_{i,t}^{\omega} &= I_{i,t-1}^{\omega} + \xi_{i,t} + r_{i,t}^{\omega} + y_{i,t}^{\text{in}} - y_{i,t}^{\text{out}} + \bar{Q}_{i,t}^{\text{arr}} \\ &\quad + \text{In}_{i,t}^{\omega} - \text{Out}_{i,t}^{\omega} - \eta_{i,t}^{\omega} - s_{i,t}^{\omega}, \\ &\quad \forall i \in \mathcal{H}^{\text{hub}}, t \in \mathcal{T}, \end{aligned} \quad (14)$$

$$\begin{aligned} I_{i,t}^{\omega} &= I_{i,t-1}^{\omega} + \xi_{i,t} + r_{i,t}^{\omega} + E_{i,t} + \bar{Q}_{i,t}^{\text{arr}} \\ &\quad + \text{In}_{i,t}^{\omega} - \text{Out}_{i,t}^{\omega} - \eta_{i,t}^{\omega} - s_{i,t}^{\omega}, \\ &\quad \forall i \in \mathcal{H}^{\text{spoke}}, t \in \mathcal{T}, \end{aligned} \quad (15)$$

$$\begin{aligned} I_{j,t}^{\omega} &= I_{j,t-1}^{\omega} + \xi_{j,t} + r_{j,t}^{\omega} + \bar{Q}_{j,t}^{\text{arr}} \\ &\quad + \text{In}_{j,t}^{\omega} - \text{Out}_{j,t}^{\omega} - \eta_{j,t}^{\omega} - s_{j,t}^{\omega}, \\ &\quad \forall j \in \mathcal{L}, t \in \mathcal{T}, \end{aligned} \quad (16)$$

$$0 \leq x_{ijk,t}^{\omega} \leq F_{ijk}^{\text{cap}}, \quad \forall (i,j,\kappa) \in \mathcal{A}, t \in \mathcal{T}, \quad (17)$$

$$0 \leq I_{n,t}^\omega \leq U_n^{\text{cap}}, \quad \forall n \in \mathcal{N}^{\text{inv}}, t \in \mathcal{T}, \quad (18)$$

$$r_{n,t}^\omega, s_{n,t}^\omega \geq 0, \quad \forall n \in \mathcal{N}^{\text{inv}}, t \in \mathcal{T}, \quad (19)$$

with $I_{n,0}$ fixed as the window initial inventory. Equations (14)–(16) are node–arc balances; the star sums (8) enforce hinterland feasibility through $\mathcal{A}_n^\pm \subseteq \mathcal{A}$. The remaining inequalities are arc capacities (17) and yard caps (18). Lease variables are unbounded above; spill s with large c^{spill} provides relatively complete recourse.

3.2.3. Success criterion for the agentic task

The agentic task is to emit a Gurobi implementation of the extensive-form DEP equivalent to (3)–(19). An expert DEP solve on the frozen smoke instance yields a known optimum $z^\star$ (approximately 1.390×10^6 for $H=4$, $|\Omega|=5$, seed 42). Critically, a solver report of OPTIMAL is *not* by itself a certificate that the emitted model is the intended DEP: an incomplete or misspecified formulation can still be feasible and yield a spurious optimum $\hat{z} \neq z^\star$. Table 1 therefore distinguishes four terminal outcomes by crossing the solver status with model correctness relative to $z^\star$.

Table 1

Outcome categories mapping solver status to model correctness.

Outcome	Solver OPTIMAL?	Model correct?
Practical Optimal	✓	✓
Spurious Optimal	✓	×
Model Infeasibility	×	×
Runtime Error	(not reached)	(not reached)

Here *model correct* means the relative gap $|\hat{z} - z^\star|/|z^\star|$ does not exceed 10^{-3} (one tenth of a percent; the experimental default; see the provenance disclosure in Section 5.4 for how this threshold was set). A pipeline run is counted as a *strict success* (SSR) iff the outcome is *Practical Optimal*: the solver reports OPTIMAL *and* the gap test against $z^\star$ passes. OPTIMAL alone with a failed gap test is classified as *Spurious Optimal* and does not count toward SSR. Injected root-cause labels for attribution experiments are defined on the same layered pipeline (data $\triangleright$ model $\triangleright$ code) and are independent of the OR model above.

4. Proposed Method

This section develops the proposed framework bottom-up. We define the agents and their interfaces (Section 4.1), compose them into a state-machine architecture (Section 4.2), and specify their communication through a shared blackboard (Section 4.3). The final two subsections present the core contribution: a Stackelberg *inspection game* for diagnosis–repair defined on the state machine (Section 4.4) and its algorithmic realization (Section 4.5). Throughout, the framework targets *layered-error* natural-language optimization — OR pipelines with a data $\rightarrow$ model $\rightarrow$ code dependency in which errors propagate strictly downstream and contaminate downstream signals; we do not claim gains for general LLM reasoning.

4.1. Agent definitions

We retain the baseline’s role-specialized agents. Let $\mathcal{A} = \{a_1, \dots, a_5\}$, with $a_1 = \text{DataEngineer}$, $a_2 = \text{ModelExpert}$, $a_3 = \text{PythonDeveloper}$, $a_4 = \text{SolverExecutor}$, $a_5 = \text{DiagnosisAgent}$. Each agent is characterized by a tuple

$$a_i = (i, \mathcal{R}_i, \mathcal{T}_i, \mathcal{B}_i, \mathcal{H}_i), \quad (20)$$

where i is a unique identifier, $\mathcal{R}_i$ the functional role, $\mathcal{T}_i$ the prompt template encoding role-specific instructions and output-format constraints, $\mathcal{B}_i$ an optional knowledge base (active only for a_2 , whose modeling decisions are domain-specialized), and $\mathcal{H}_i$ the hyper-parameter configuration (temperature, top_p, max_tokens).

The five roles decompose the modeling task along the causal order *data* $\triangleright$ *model* $\triangleright$ *code*. *DataEngineer* (a_1) parses the problem description and raw instance into structured parameters with symbol anchoring for downstream consistency; *ModelExpert* (a_2) designs the optimization model (decision variables, objective, constraints); *PythonDeveloper* (a_3) translates the formulation into executable solver code; *SolverExecutor* (a_4) runs the code

in a sandbox, validates the result against the strict-success rule, and reports status; *DiagnosisAgent* (a_5) analyzes any failure and drives repair.

Each correctable agent exposes two maps: a *forward* map a_i^+ that produces its nominal output, and a *backward* (repair) map a_i^- that, given an error signal, either accepts responsibility and emits a corrected output or rebuts. Agent a_i receives an input assembled from prior outputs,

$$x^{(i)} \leftarrow f_i(y^{(0)}, y^{(1)}, \dots, y^{(i-1)}), \quad y^{(i)} \leftarrow a_i^+(x^{(i)}), \quad (21)$$

where f_i selects the necessary dependencies. The backward map is invoked only on the agent implicated by diagnosis (Section 4.4).

4.2. Multi-agent architecture and state-machine orchestration

The agents of Section 4.1 are composed into an extended finite-state machine rather than a free-form graph. The pipeline is compiled as

$$M = (\mathcal{Q}, q_0, \Delta, S, K), \quad (22)$$

where $\mathcal{Q}$ is a finite set of control states (the graph nodes), q_0 the unique entry state, Δ the transition map, S the shared blackboard (Section 4.3), and K the transition budget. The state set is

$$\mathcal{Q} = \{ q_{DE}, q_{ME}, q_{KL}, q_{PD}, q_{SE}, q_{DG} \} \cup \{ q_{DE}^-, q_{ME}^-, q_{PD}^- \}, \quad (23)$$

comprising six forward states — one per agent in $\{a_1, \dots, a_5\}$ plus a dedicated KNOWLEDGELOADER state — and three backward (repair) states, one per correctable agent. The entry state is $q_0 = q_{DE}$ and the lone terminal is END. Figure 1 depicts M .

Transitions are pure functions of the blackboard. The transition map Δ is realized by five routing functions, each a pure map $S \rightarrow \mathcal{Q}$:

- δ_{ME} : after ModelExpert, branch to KNOWLEDGELOADER iff it requests not-yet-loaded modules, else to PythonDeveloper;
- δ_{KL} : after KnowledgeLoader, return to ModelExpert iff new knowledge was added, else proceed to PythonDeveloper;
- δ_{SE} : after SolverExecutor, terminate on success or budget exhaustion, otherwise route to DiagnosisAgent;
- δ_{DG} : after DiagnosisAgent, route to the backward state of the implicated agent (the diagnosis verdict);
- δ^- : after a backward step, either resume the pipeline downstream from the repaired agent, re-enter diagnosis, or terminate — depending on whether the repair was accepted and which agent was repaired.

The retry budget K (`max_retries`) bounds the number of diagnosis–repair cycles, i.e., it is the FSM’s transition budget over the diagnostic sub-graph.

Domain knowledge is progressive scaffolding, not the contribution. ModelExpert does not receive a dump of every domain file, nor a RAG retriever. It first sees a *catalog* (module name + short description); it may then request modules by name; KNOWLEDGELOADER injects only those full texts and returns control for refinement. This on-demand path is required for correct TSLP modeling but is orthogonal to the Stackelberg inspection game: both features plug into the same FSM without owning each other’s logic.

The FSM is the stage-game apparatus, not scaffolding. The state machine is the structural substrate of the contribution because it is the object on which the repair game is defined. Its states are stage-game states, and — once the repair game is in play — the transitions δ_{DG} and δ^- become functions of the players’ strategy profile (the inspector’s committed inspection policy and the inspectee’s verified response; Section 4.4) rather than of a fixed heuristic. Treating M as the stage-game apparatus is what lets the commitment order carry *causal* meaning and what supports the proposition in Section 4.4; without this formalization, the FSM would be indistinguishable from generic graph orchestration.

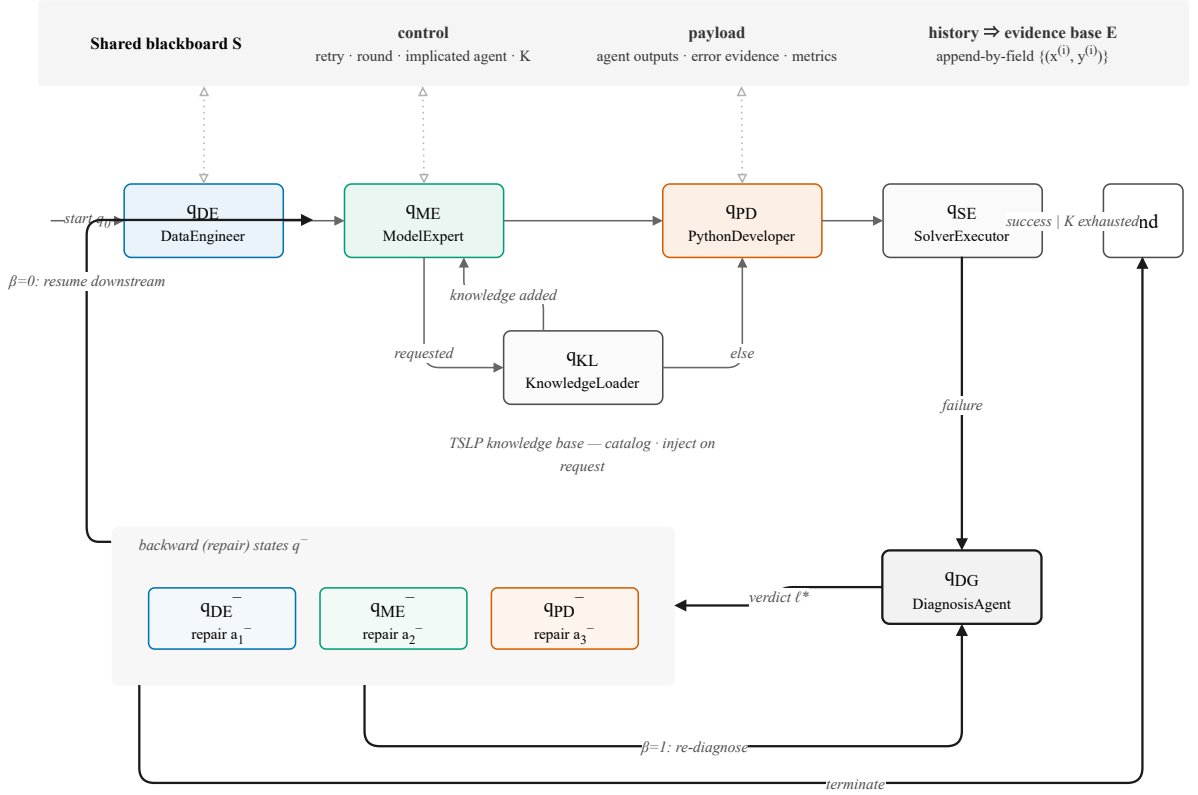


Figure 1: The extended finite-state machine $M = (Q, q_0, \Delta, S, K)$. Node colors mark the causally ordered correctable layers data $\triangleright$ model $\triangleright$ code. q_{KL} injects, on demand from a catalogued TSLP knowledge base (two-stage structure; stage-1 sea and stage-2 inland decisions [D1]/[D2]; network nodes; demand scenarios Ω ; exogenous supply ξ ; data layout), the modules ModelExpert requests. Thin gray arrows: the forward chain. Bold dark arrows: the diagnosis–repair cycle — the diagnosis verdict ℓ^* selects the backward state, and the return transition δ^- routes on the executed Stackelberg verdict (Section 4.4): resume downstream of the repaired agent ($\beta=0$), re-enter diagnosis ($\beta=1$), or terminate. Dotted stubs: every state reads from and writes to the shared blackboard S only, whose append-by-field history constitutes the evidence base $\mathcal{E}$ of the inspection game.

4.3. Inter-agent communication via a shared blackboard

Agents never exchange messages pairwise. All communication is mediated by a single typed blackboard S (realized as the AgentState schema): every node reads its inputs from S and writes its outputs back to S , and the transition functions Δ route solely on S 's contents. This *shared-state, no-direct-links* design has three consequences.

Control and payload on one schema, separated by reader. S carries both *control state* and *data payload*. Control fields — retry count, the currently implicated agent, a resolved flag, the diagnosis mode, the round counter, the retry budget — drive Δ and are the only fields a routing function inspects. Data fields — each agent's output, the assembled error evidence, execution metrics, the loaded knowledge modules, and the ground-truth objective — carry the pipeline's content and are read by the agent that fires next. Colocating these concerns on one schema while separating their readers is what makes the FSM's transitions both data-aware and cheap to reason about.

A consistent, fixed data view. A deterministic preprocessor runs once, before any LLM agent fires, and writes a canonical view of the raw instance into S . Every downstream agent therefore reasons over an identical data basis, eliminating a class of data-parsing races that would otherwise masquerade as modeling or coding errors and confuse the diagnosis mechanism.

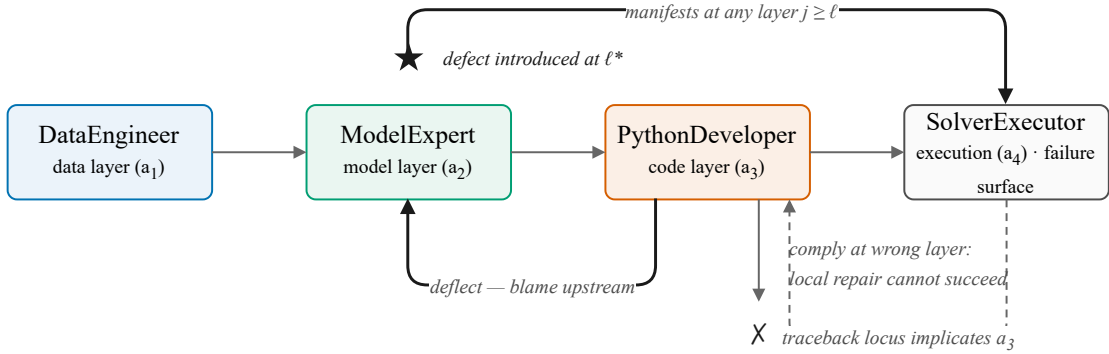
Append-by-field history. Communication is append-by-field rather than overwrite: a backward step writes a corrected output together with a resolved flag without erasing the prior forward outputs. The diagnosis mechanism can therefore inspect the full input–output history $\{(x^{(i)}, y^{(i)})\}$ when attributing a failure. This history is the evidence base $\mathcal{E}$ on which the Stackelberg game of Section 4.4 is defined, and its completeness is what makes blame attributable rather than heuristic.

4.4. The Stackelberg diagnosis–repair game as an inspection game

We cast the diagnosis–repair stage as a Stackelberg game on the FSM multi-agent pipeline. The DiagnosisAgent moves first and commits — observably — to an inspection policy; the accused correctable agents then best-respond. Leader commitment plus follower response matches the pipeline’s control structure: the orchestrator already routes through diagnosis before any repair is applied, so the natural extensive form is sequential rather than simultaneous classification.

4.4.1. Causal layering of defects and the deflection incentive

Let the correctable core of the pipeline be the chain $a_1 \triangleright a_2 \triangleright a_3$ (DataEngineer $\triangleright$ ModelExpert $\triangleright$ PythonDeveloper), i.e. $data \triangleright model \triangleright code$, where $\triangleright$ denotes “is a causal prerequisite of.” We assume *strict causal layering of defects*: a defect introduced at layer $\ell \in \{1, 2, 3\}$ may manifest at any layer $j \geq \ell$, but a defect at layer ℓ cannot be the root cause of a symptom that requires only layers $< \ell$ to be well-formed. Consequently, the observation of a downstream failure (e.g. an infeasible solve, a wrong objective, or a traceback into solver-facing code) is consistent with several root-cause layers, and localizing the true root cause ℓ^* from the failure evidence $\mathcal{E}$ alone is fundamentally ambiguous: $\mathcal{E}$ records the *symptom locus* (often a_3), not a certified identity of ℓ^* . In particular, one cannot read the root-cause layer off the SolverExecutor’s failure surface alone: a Python traceback or crash locus typically marks where the pipeline *broke*, most often in generated code, rather than where the defect was *introduced*. The failure surface therefore motivates an *inspection* protocol rather than a one-shot classification of the traceback locus. Figure 2 sketches this structure.



An innocent implicated layer strictly prefers to deflect; a guilty layer may also deflect — diagnosis becomes a game, not a classification.

Figure 2: Causal layering of defects and the deflection incentive (Section 4.4.1). A defect introduced at layer ℓ^* (star) propagates strictly downstream and may manifest at any layer $j \geq \ell$; the SolverExecutor failure surface records the symptom locus (often generated code), not the certified root cause. Reading the traceback locus therefore implicates a_3 heuristically (dashed arrow). An implicated-but-innocent layer strictly prefers to *deflect* (bold arrow): a local repair cannot succeed and only consumes tokens, so complying at the wrong layer fails re-solve (X). A guilty layer may likewise deflect to dodge the repair cost — this misalignment is what turns diagnosis into an inspection game rather than a classification.

Causal layering of defects does not merely make attribution ambiguous; it gives the accused agents *misaligned private incentives*, which is what turns diagnosis into a game rather than a classification. Consider an agent at layer ℓ that a failure has implicated. Two facts hold simultaneously: (i) repair is costly — a backward step re-generates the agent’s output, consumes tokens, and risks injecting a fresh defect; and (ii) if $\ell \neq \ell^*$ (the layer is innocent, merely downstream of the true cause), then *no* local repair at ℓ can make the solve pass, because by strict causal layering of defects the true defect lies upstream. An innocent implicated agent therefore strictly prefers to *deflect* — to rebut the accusation and point blame upstream — rather than to accept a repair that wastes effort and cannot succeed.

Symmetrically, even a guilty layer, if it could avoid the repair cost by deflecting, would be tempted to do so. The accused agents are thus not neutral truth-seekers whose objective coincides with the diagnostician’s; they are self-interested inspectees. Resolving this requires an inspector that *commits* to verification, which is what Section 4.4.2 constructs.

This incentive structure is exactly that of an inspectee facing an inspector. We accordingly specialize the stage game to a Stackelberg *inspection game* (Avenhaus et al., 2002; Maschler, 1966), a canonical class of leader–follower games in which an inspector commits to a monitoring policy and an inspectee best-responds. The diagnosis agent is the inspector (leader); the accused correctable agents are inspectees (followers); and the inspector’s first-mover *commitment* to a causal-order, executed-refutation policy is what induces truthful attribution as the inspectees’ best response. Table 2 maps the inspection-game elements to the pipeline and contrasts them with non-committed, single-judge protocols.

Table 2

Mapping of Stackelberg inspection-game elements to the diagnosis–repair stage. Right column: what non-committed / single-judge protocols provide instead.

Inspection-game element	Concretization in our mechanism	Non-committed / single-judge protocols
Inspector / leader (moves first, commits)	DiagnosisAgent commits an inspection policy σ : probe order + executed refutation	One LLM call picks a suspect
Inspectees / followers (best-respond)	Accused agents a_1, a_2, a_3 : comply-and-repair or deflect-upstream	Accused agent self-checks in isolation
Committed policy σ	Causal-order probing + verify by real re-solve (no cheap talk)	Fixed heuristic prior + single confidence
Conflict of interest	Innocent layer deflects to avoid wasted repair / wrongful blame	(assumed absent)
Inspector payoff	Correct root-cause localization at fewest rounds/tokens	(undefined)
Inspectee payoff	Avoid wrongful blame and wasted repair; fix only if truly at fault	A confidence score
Induced outcome	Guilty layer complies; innocent deflection is verified	Confidence-routed re-accusation
Stage-game apparatus	The LangGraph FSM (states = stage-game states)	(treated as plumbing)

4.4.2. Game formulation

A diagnosis–repair stage game G is instantiated whenever Δ routes from q_{SE} to q_{DG} (the solver has failed and the budget is not exhausted). The game is played on the current blackboard $S_t \in \mathcal{S}$ and has the following structure.

Players. An *inspector* I (the leader) and a set of *inspectees* (the followers). The inspector is the DiagnosisAgent a_5 , operating as an independent examiner. The inspectees are the correctable agents a_1, a_2, a_3 ; in a given probe the active inspectee is the agent a_{ℓ} at the currently probed layer ℓ .

Timing. Play is Stackelberg: the inspector first commits to an observable inspection policy; the active inspectee then responds; the inspector finally settles the response by verification. The probing order ω follows the causal order $a_1 \triangleright a_2 \triangleright a_3$: the inspector probes the seeded suspect layer first and, when a layer is cleared, descends to the next causal layer. Aligning commitment order with causal order makes the first move causally meaningful (an upstream probe is settled by re-solve before the inspector descends) rather than a coin flip among observationally equivalent layers.

Strategies. The inspector’s strategy is an *inspection policy* $\sigma = (\omega, \nu)$ fixed *before* any inspectee responds: (i) a probing order ω over the layers, and (ii) a verification rule ν that resolves every response by *executed refutation* rather than by accepting the inspectee’s word. When probed at layer ℓ , the inspectee chooses $r \in \{\text{COMPLY}, \text{DEFLECT}\}$. To COMPLY is to accept responsibility and emit a concrete repair $h \in \mathcal{H}_{\ell}$; to DEFLECT is to rebut the accusation and assert that the root cause lies upstream. Given evidence $\mathcal{E}$ from the blackboard — stack trace, solver log, implicated code,

model definition, data structures, and pipeline input–output pairs $\{(x^{(i)}, y^{(i)})\}$ — the active inspectee emits

$$(r, h) \leftarrow a_{\hat{\ell}}^{-}(x^{(\hat{\ell})}, y^{(\hat{\ell})}, \mathcal{E}), \quad r \in \{\text{COMPLY}, \text{DEFLECT}\}, \quad (24)$$

where, if $r = \text{COMPLY}$, h carries a justified repair, and if $r = \text{DEFLECT}$, h carries an upstream rebuttal.

Verification. Under its committed rule v , the inspector does not take the response on trust. It selects and *executes* the cheapest decisive test,

$$\tau^* \leftarrow \arg \min_{\tau \in \mathcal{T}(r, h)} \text{cost}(\tau) \quad \text{s.t. } \tau \text{ is decisive}, \quad (25)$$

— a re-solve after applying h when $r = \text{COMPLY}$, or a targeted re-solve isolating the named upstream layer when $r = \text{DEFLECT}$ — and issues a verdict

$$v = (\beta, \hat{\ell}^*) = I(r, h, \tau^*, S_t), \quad (26)$$

where $\beta \in \{0, 1\}$ records whether the probed layer is *cleared* ($\beta=1$: a compliance whose re-solve still fails, or a deflection the inspector upholds) or *confirmed* ($\beta=0$: a compliance whose re-solve passes the strict-success rule), and $\hat{\ell}^*$ is the updated root-cause estimate. Executing the test supplies a grounded signal: neither a false confession nor a dishonest deflection survives. The FSM transition δ^- routes on this verdict: if $\beta = 0$, the repair in h is committed and Δ resumes downstream of $a_{\hat{\ell}}$; if $\beta = 1$, Δ re-enters diagnosis and the inspector probes the next causal layer, until confirmation or budget exhaustion (K).

Payoffs. Because LLMs admit no stable, observable utility function, we do *not* claim that an LLM “feels” a payoff. Instead we define numeric payoffs that the orchestrator optimizes and treat each LLM call as a *black-box best-responder*, analyzing the resulting trajectory. The inspector’s payoff rewards correct localization at low cost,

$$u_I(\sigma) = \mathbf{1}[\hat{\ell}^* = \ell^*] - \lambda_I \text{cost}(\sigma), \quad (27)$$

while the inspectee at layer ℓ , responding r , has the misaligned payoff

$$u_{\ell}(r; \sigma) = b \cdot \mathbf{1}[r=\text{COMPLY} \wedge \text{re-solve passes}] - c_{\text{rep}} \mathbf{1}[r=\text{COMPLY}] - c_{\text{blame}} \mathbf{1}[\text{held responsible} \wedge \ell \neq \ell^*], \quad (28)$$

where $b > 0$ is the reward for actually resolving the failure, $c_{\text{rep}} > 0$ the cost/risk of a repair, $c_{\text{blame}} > 0$ the penalty for carrying blame that does not fix the solve, $\text{cost}(\cdot)$ is token/round cost, and $\lambda_I \geq 0$ trades accuracy against budget. Eq. (28) makes the conflict explicit: the inspectee is rewarded only when it is *both* at fault *and* repairs, and is otherwise penalized for complying, so its interests diverge from the inspector’s whenever $\ell \neq \ell^*$. The ground-truth label ℓ^* enters only the *evaluation* of payoffs, never any agent’s prompt, so the game remains well-defined at inference time; equilibria are properties of the orchestrator-defined game, while each LLM is a sampled best-responder within it.

4.4.3. Induced truthful attribution

The value of the inspector’s first move is that its committed executed-refutation rule v reshapes the inspectee’s best response — the classical inspection-game deterrence effect. Consider the two cases. (i) *Innocent inspectee* ($\ell \neq \ell^*$): by strict causal layering of defects a local repair cannot make the solve pass, so COMPLY yields $-c_{\text{rep}}$ (no reward b , and it will still be seen not to fix the solve), whereas a truthful DEFLECT avoids the repair cost and is upheld by the inspector’s targeted re-solve. Deflection strictly dominates — and, crucially, only a *truthful* deflection survives v , since a dishonest one is refuted by re-solve. (ii) *Guilty inspectee* ($\ell = \ell^*$): deflecting cannot succeed, because every upstream probe the inspector runs under causal order passes verification and the probe returns to ℓ^* having only consumed rounds; complying and repairing yields $b - c_{\text{rep}}$. Whenever $b > c_{\text{rep}}$ (a resolved solve is worth more than the repair), COMPLY is the guilty layer’s best response. Hence the committed policy makes *truthful attribution* — the guilty complies, the innocent deflects honestly — the induced best-response profile. It is the inspector’s *commitment*, not any assumed benevolence of the agents, that aligns individually optimal responses with correct root-cause localization. Without commitment, deflection is cheap talk and truthful attribution is not incentive-compatible.

Information assumption. The best-response analysis above assumes the inspectee observes its own type (ℓ versus ℓ^*) before responding. For LLM inspectees this is a behavioural idealization, not an implementation fact: a

comply/deflect output is one model reading the blackboard evidence. The assumption is grounded exactly when the defect is visible in the layer’s own artifact — a model-layer fault is inspectable in the emitted formulation, so evidence-reading behaviour approximates the informed best response — and defeated by rank-invisible semantic data faults. The measured comply/deflect profile (Section 5.7) is therefore read as evidence of how closely the implemented agents approximate this intended equilibrium, not as a proof of it.

4.4.4. Proposition

Proposition 1 (Causal-order inspection lowers mis-attribution)

Premises. (A1) Strict causal layering of defects (Section 4.4.1): a defect at layer ℓ cannot cause a symptom that requires only layers $<\ell$. (A2) No absorption: a repair emitted by an *innocent* layer cannot pass the executed re-solve. (A3) Repair sufficiency: a complying repair emitted by the *guilty* layer passes the executed re-solve.

Claim. Under A1–A3 and an inspector committed to a causal-order, executed-refutation policy, each probe is *decisive* — it confirms or clears the probed layer — and cleared layers are innocent, so the true layer is never eliminated. Relative to a single-judge round, which produces an unverified point hypothesis and therefore changes no layer’s status, every cleared probe strictly shrinks the viable root-cause set. Formally, with V_{sim} and V_{ins} the sets of layers consistent with the failure observation after one round of, respectively, single-judge play (hence the full observationally-equivalent cone) and causal-order inspection, $V_{\text{ins}} \subseteq V_{\text{sim}}$ with $\ell^* \in V_{\text{ins}}$, and the inclusion is strict whenever a layer $\ell < \ell^*$ is probed and cleared.

Proof. An innocent probed layer cannot pass the re-solve by A2: its compliance is refuted and its truthful deflection upheld, and either way it is cleared — so only innocent layers are ever cleared. For the guilty layer, compliance is the best response (Section 4.4.3) and by A3 its repair passes, so it is confirmed and ℓ^* is never eliminated. A single-judge round executes no test, so no layer changes status and V_{sim} is the full observationally-equivalent cone; inspection removes from it every cleared layer. $\square$

Status of the premises. A1 is the structural assumption of Section 4.4.1; A2 and A3 are empirical regularities of the implemented repair path, not axioms, and the measured campaign prices both. Absorption (A2 failure) occurs on eight reverse-arm boards, where an innocent code layer’s free regeneration silently drops the upstream constraint and passes the re-solve; repair insufficiency (A3 failure) occurs on nine causal boards, where the guilty layer’s complying repair is overturned and the search moves past the true layer (both quantified in the Exp-A mechanism findings, Section 5.5). The proposition delimits what commitment buys when the repair path cooperates; the funnel measures how often it does.

4.5. The diagnosis–repair algorithm

Algorithm 1 unifies the architecture and the repair game. The FSM drives the forward pass; on failure it enters the inspection stage, where the inspector commits to a causal-order, executed-refutation policy and probes layers in causal order — each probe eliciting an inspectee response and settling it by re-solve — until a layer is confirmed or the budget K is spent. The transition map Δ is exactly the five routing functions of Section 4.2, with the return edge out of the backward state set by the executed verdict v .

Relation to the baseline. The forward chain, the blackboard, and the retry budget are inherited unchanged. The replacement is localized: the single-judge diagnosis call and its confidence-routed return are substituted by the committed inspection stage, and the return edge of δ^- is driven by the executed verdict β rather than by a confidence. This locality is deliberate — it isolates the contribution (the inspection game) from the scaffolding (the FSM) and makes the commitment-order ablation, which randomizes the order in which layers are probed, the decisive experiment: if a permuted order performs as well as causal order, the thesis is falsified.

5. Experiments

The campaign tests whether a committed inspection policy improves *verifiable* root-cause attribution under layered faults—and whether it can match a strong single-judge LLM on naming accuracy—rather than whether any protocol can eventually repair a failed solve.

Algorithm 1 FSM-Stackelberg diagnosis–repair (inspection game)

Require: Problem $\mathcal{P}$; agents $\{a_1, a_2, a_3, a_4\}$; inspector role a_5 ; FSM $M = (\mathcal{Q}, q_0, \Delta, S, K)$; causal order $a_1 \triangleright a_2 \triangleright a_3$

Ensure: Result $Y \in \{\text{success}, \text{failure}\}$

```

1:  $S \leftarrow \text{PREPROCESS}(\mathcal{P})$ ;  $k \leftarrow 0$ ;  $Y \leftarrow \perp$ 
2: while  $Y = \perp$  do  $\triangleright$  drive the FSM
3:   // Forward chain:  $q_{\text{DE}} \rightarrow q_{\text{ME}} \rightarrow \{q_{\text{KL}}\} \rightarrow q_{\text{PD}} \rightarrow q_{\text{SE}}$ 
4:   for  $i = 1$  to 4 do
5:      $y^{(i)} \leftarrow a_i^+(S)$ ; update  $S$ ; advance  $\Delta$ 
6:   if  $y^{(4)}.\text{STATUS} = \text{OPTIMAL}$  then
7:      $Y \leftarrow \text{success}$ 
8:   else if  $k \geq K$  then
9:      $Y \leftarrow \text{failure}$ 
10:  else
11:    // Inspection stage at  $q_{\text{DG}}$ : inspector commits  $\sigma = (\text{causal order}, \text{executed refutation})$ 
12:     $\hat{\ell} \leftarrow \text{PRIOR}(y^{(4)}.\text{STATUS})$   $\triangleright$  seed probe order
13:    repeat
14:       $(r, h) \leftarrow a_{\hat{\ell}}^-(S)$   $\triangleright$  inspectee best-responds: comply-repair or deflect
15:       $\tau^* \leftarrow \arg \min_{\tau \in \mathcal{T}(r, h)} \text{cost}(\tau)$  s.t. decisive
16:       $(\beta, \hat{\ell}^*) \leftarrow a_5.\text{VERIFY}(r, h, \tau^*, S)$   $\triangleright$  executed refutation (re-solve)
17:       $k \leftarrow k + 1$ 
18:      if  $\beta = 0$  then  $\triangleright$  layer
19:         $\text{confirmedcommit } \tilde{y} \in h$ ; resume  $\Delta$  downstream
20:         $\hat{\ell} \leftarrow \text{NEXTCAUSALLAYER}(\hat{\ell})$   $\triangleright$  layer cleared: descend causal order
21:    until  $\beta = 0 \vee k \geq K$ 
21: return  $Y$ 

```

Claim and mechanism under test. We do not claim that Stackelberg inspection is uniquely able to name ℓ^* from a stack trace: at failure time every method sees the same blackboard, so a capable single-judge LLM may match first-shot accuracy. The intended footing is the conjunction of (i) accuracy that is at least competitive with single-judge diagnosis, (ii) *verifiable* attribution via commitment plus executed refutation, and (iii) cost/audit properties under the same budget K . Achieving (i) requires an *evidence-informed* commitment: the leader forms a *layer ranking* from the stack and agent histories (ranks or votes—not LLM-reported probabilities, which are poorly calibrated), commits ω , then probes with executed refutation. Status-prior-only ω is treated as a preliminary apparatus, not the final proposed policy.

Pre-registered expectations. Each experiment below states an *expected conclusion* fixed before the corresponding redesigned runs. The campaign is organized as three blocks: commitment-order ablation, a unified baseline comparison (internal and external protocols), and mechanism diagnostics. Tables marked *measured* report the evidence-informed redesign (omega_source=evidence_rank, hybrid rank under the force-zero plant); tables marked *preliminary* are from the earlier status-prior ω implementation and do not substitute for the redesign. External Debate/Reflexion ran later under the same engine and snapshot protocol (measured below); Exp-C remains unrun at the time of writing.

5.1. Setup

Data. All methods use the demand-uncertainty TSLP ECR model of Section 3 on the exported dataset prob_ts1p_ecr_demand. We start from the smoke instance smoke_H4_Omega5 and extend to a small family of horizon/scenario configurations from the same generator. The forward pipeline, blackboard schema, strict-success rule (Table 1), and sandbox solver path are shared. An end-to-end smoke run on smoke_H4_Omega5 with progressive knowledge injection terminated in *Practical Optimal* ($\hat{z} \approx z^* \approx 1.390 \times 10^6$, relative gap below the 10^{-3} threshold): the solver’s OPTIMAL status was therefore *not* a Spurious Optimal under the taxonomy above.

Model and budget K . Provider and model are held fixed within each comparison block and reported explicitly. Every method receives the same retry budget K (max_retries); the single declared exception is the Reflexion budget-tilted challenger sweep at $K=5$ (Section 5.6).

Injection protocol. Ground-truth root causes $a^* \in \{a_1, a_2, a_3\}$ are obtained by controlled injection: a defect is planted at one of the three layers (DataEngineer, ModelExpert, or PythonDeveloper) so that the surface symptom often appears downstream (e.g., a traceback or crash locus in generated code, or a feasible-but-wrong optimum). The measured grids use three plants, one per layer: `me_force_zero_sea` ($a^*=\text{ModelExpert}$), `pd_comment_out_balance` ($a^*=\text{PythonDeveloper}$; balance constraints commented out of generated code), and `de_swap_demand_supply_source` ($a^*=\text{DataEngineer}$; a contract-valid semantic swap of demand and supply catalog sources). This matches the motivation in Section 4.4 that failure evidence records a symptom locus, not a certified identity of ℓ^* . The label a^* is used only for evaluation, never in agent prompts.

Fairness. At failure time every method reads the same blackboard (solver status, stack trace, agent input–output history). Repair uses the same agent `backward_step` maps and the same re-solve verification path; methods differ only in how they select or order the implicated layer. No method receives privileged tools beyond its diagnosis policy. In snapshot mode (below) the same-blackboard promise is exact by construction: the failure state is frozen once per seed and all arms resume it verbatim.

5.2. Methods compared

- **(A) Proposed / ablation family.** `diagnosis_mode=stackelberg` with committed policy $\sigma = (\omega, \nu)$ and ν fixed as executed refutation. The leader forms an evidence-informed layer ranking (no confidence weights), then commits a probing order $\omega \in \{\text{causal}, \text{reverse}, \text{random}\}$ obtained by aligning that ranking with (or permuting against) the `data>model>code` skeleton.
- **(B) Internal baselines** (Exp-B, same table as C). (i) *adversarial*: single-judge LLM accusation plus accused-agent self-check; (ii) *sequential*: fixed reverse-order backward chain without an inspection commitment.
- **(C) External verification baselines** (Exp-B). (i) *Debate* (Du et al., 2023): multiple diagnosis personas argue for a fixed number of rounds; terminal positions are aggregated by majority vote (no LLM-reported confidence weights; ties broken by the status prior), then the accused layer enters the shared repair/re-solve path. (ii) *Reflexion* (Shinn et al., 2023): single-judge-style accusation augmented with verbal reflections after failed diagnose–repair attempts, conditioned upon in later diagnosis rounds, again sharing the same repair/re-solve machinery. Internal and external protocols answer one baseline question and are reported together in Section 5.6; execution may stage (B) before (C) for cost control.

5.3. Metrics

The primary metric is *verified* root-cause attribution accuracy $\mathbf{1}\{\hat{a} = a^*\}$ where $\hat{a}$ is the layer supported by executed refutation (not merely last-comply credit), optionally stratified by injected origin and solver-status surface. For commitment-order contrasts we also report the first-probe hit $\mathbf{1}\{\omega_1 = a^*\}$ — a *commitment-side* quantity: under the committed policy the engine routes every probe to the first not-yet-cleared layer of ω , so the executed first probe coincides with ω_1 by construction and the metric reads out whether the committed order — rank-aligned (causal), reversed, or shuffled (random) — targeted the true layer. Its outcome-side complement is *first-probe confirmation*: the probed guilty layer complies and its repair passes the executed re-solve at the first probe (equivalently, verified attribution at $K=1$). Confirmation requires alignment *and* repairability, and the gap between the two metrics locates the repair path’s failure modes. Secondary metrics are SSR@ K , probe rounds, and loop tokens (total minus the frozen forward prefix, Section 5.4). SSR counts only *Practical Optimal* outcomes in Table 1; a bare OPTIMAL that fails the z^* gap test (*Spurious Optimal*) does not contribute. For Exp. 5.7 we additionally record deflect rates, the fraction of compliances and deflections overturned by executed refutation, and analysis-mode episode payoffs (u_L, u_F) from Section 4.4.2. Highest SSR alone is not a sufficient claim: under strong models many protocols may eventually repair; the scientific contrast is verifiable attribution under commitment at competitive naming accuracy.

5.4. Result interpretation and provenance

Reading results against expectations. Pre-registered expectations are the reading grid for every table that follows: an outcome *supports*, *does not support*, or *falsifies* the registered expectation, and an unsupported expectation revises the design before further experimental spend rather than accumulating post-hoc explanations. The campaign has already exercised this rule once. The status-prior implementation failed its attribution expectations outright—last-comply attribution was 0/5 for every method in the baseline comparison (Table 8), and no order exceeded 1/5 in

the ablation (Table 6)—and the recorded response was a redesign of the commitment apparatus (evidence-informed ranking plus verified attribution accounting), not a re-labelling of the old runs; the status-prior tables are retained below only as preliminary context. The grid was run under sequential disclosure: an $n=3$ feasibility pass, an interim inspection at $n=5$, an expanded $n=20$ campaign, and a further densification to $n=30$, all with unchanged expectations; freeze-phase attempts that never produce a failure blackboard (data-contract rejections by the deterministic validator, or plants not expressed in the generated model) are reported as forward variance and replaced with fresh seeds, never silently dropped. Confirmatory claims below are restricted to the clauses the expanded grid can power (first-probe); verified-attribution contrasts that remain non-significant are stated as such. SSR is never interpreted in isolation from attribution (Section 5.3).

Provenance and implementation versions. Every measured cell is produced by a single frozen implementation. Each run writes a machine-readable manifest (configuration, provider and model, terminal metrics) and a per-node event log (tokens, durations, comply–deflect responses); tables aggregate these records, and the manifests and event logs—not any intermediate summary file—are the authority for every number reported. Implementation versions are declared rather than silently mixed. The measured tables below were produced under the hardened data layer (deterministic catalog and validator constraining the data-access guide in forward and backward passes), re-established after a no-plant end-to-end gate terminated in *Practical Optimal* (measured relative gap 0.0%, far below the 10^{-3} threshold), on provider DeepSeek / `deepseek-flash` (thinking disabled, temperature 0), and in *snapshot mode*: the failure blackboard is frozen once per seed at the solver→diagnosis boundary and every arm resumes the identical board, so order arms are paired on the same failure state by construction; token costs are *loop tokens* (total minus the frozen forward prefix). Earlier DashScope $n=3$ cells predate this campaign and are neither pooled nor compared numerically; each table carries the campaign, and hence the implementation freeze, it was produced under. One classification parameter was tightened during analysis and is disclosed here rather than silently applied: the strict-success gap threshold moved from 10^{-2} to 10^{-3} . The choice is data-driven rather than arbitrary: across the 316 measured runs with a computable gap the distribution is sharply bimodal—exactly 0.0000% for 184 runs and at least 1.01% for 129, with only three cells in between (0.03%, 0.53%, 0.99%)—and the instances are LPs whose deterministic equivalents two exact solvers agree on to $\sim 10^{-9}$ relative, so the in-between cells are real model deviations rather than solver noise. All tables below were recomputed offline from the run manifests—the authority for every number—under the tightened rule; exactly two cells reclassify from Practical Optimal to Spurious Optimal (one Debate external cell at 0.53%, one random-order ablation cell at 0.99%), both in arms competing with causal Stackelberg, and no directional conclusion changes.

5.5. Exp-A: Commitment-order ablation

Question. Does the inspector’s committed probing order carry information under layered faults? Equivalently: holding v and K fixed, is causal-order commitment better than reverse or random commitment on first-probe hit and verified attribution?

Design. All cells use `diagnosis_mode=stackelberg` with evidence-informed ranking at commit time. We vary only $\omega \in \{\text{causal}, \text{reverse}, \text{random}\}$ on the injection grid of Section 5.1. Causal ω rotates the data>model>code skeleton so that the top-ranked layer is probed first; reverse and random break that alignment (and, per the redesign spec, do not reuse the rank tip), isolating order from naming skill.

Expected conclusion. Causal ω yields strictly higher first-probe hit and verified attribution than reverse, and at least weakly higher than random, at comparable token cost. If causal $\approx$ random/reverse, Proposition 1 carries no empirical bite and the ranking/alignment design is revised before baselines.

Measured results (evidence-informed; snapshot mode; $n=30/\text{order}$). Table 3 reports the expanded ablation on `smoke_H4_0omega5` with plant `me_force_zero_sea`, $K=3$, provider DeepSeek / `deepseek-flash`, and `rank_method=hybrid` (the force-zero smell tips model_expert without inventing a second diagnosis mechanism). Causal commitment attains first-probe 30/30 versus 15/30 under random and 0/30 under reverse (paired exact McNemar: 30:0 discordant pairs, $p = 1.9 \times 10^{-9}$, and 15:0, $p = 6.1 \times 10^{-5}$); the registered first-probe clause is strongly supported. Verified attribution is 14/30 causal, 13/30 random, 7/30 reverse; the causal-versus-reverse contrast is significant (8:1 discordant pairs, $p = 0.039$), and causal versus random remains a near-tie (5:4, $p = 1.0$)—the weak clause is met as a tie. Two mechanism findings qualify these counts. First, first-probe naming is commitment-side

Table 3

Exp-A commitment-order ablation — **measured** (evidence-informed; snapshot mode; $n=30/\text{order}$; $K=3$; hybrid rank; DeepSeek deepseek-flash; plant `me_force_zero_sea`; loop-token accounting). Primary: first-probe hit and verified attribution.

ω	First-probe hit	Verified attr.	SSR@ K	Mean loop tokens
causal	30/30 (1.00)	14/30 (0.47)	21/30	0.64×10^5
random	15/30 (0.50)	13/30 (0.43)	17/30	0.67×10^5
reverse	0/30 (0.00)	7/30 (0.23)	15/30	0.74×10^5

by construction (Section 5.3): the executed first probe equals ω_1 on every board, verified event-by-event against the logs, so the hit counts measure which orders targeted the guilty layer — the hybrid rank names the model layer on all 30 boards, and exactly the 15 random shuffles that placed it first hit. The outcome-side quantity is the confirmation funnel: of causal’s 30 aligned first probes, 14 confirm (guilty-layer comply, re-solve passes); the remaining 16 split into 9 whose model-layer repair is overturned by the re-solve and the run ends in terminal failure (final relative gaps 1.7–55.8%), and 7 where the model repair is correct but the downstream code regeneration injects a fresh defect, the next probe credits the code layer, and the run terminates *Practical Optimal* without verified attribution — the whole of causal’s SSR margin (21=14+7). Random mirrors the funnel on its aligned half (9 of 15 confirm; the other 6 are overturned terminal failures), and reverse exposes the converse hazard: on 8 boards an *unaligned* first probe absorbs the model-layer fault outright — the regenerated code silently drops the injected constraint, the re-solve passes, and the credit settles on the code layer; these eight boards are more than half of reverse’s SSR (8 of 15) and none is a verified attribution. Second, 4 of the random arm’s 13 verified attributions start from *unaligned* shuffles and still reach the guilty layer by executed refutation within K : alignment raises the chance of early verified attribution, but multi-round search under executed refutation is a genuine second route. Token costs are of the same order across orders (causal marginally lowest). Figure 3 summarizes the ablation and the alignment audit.

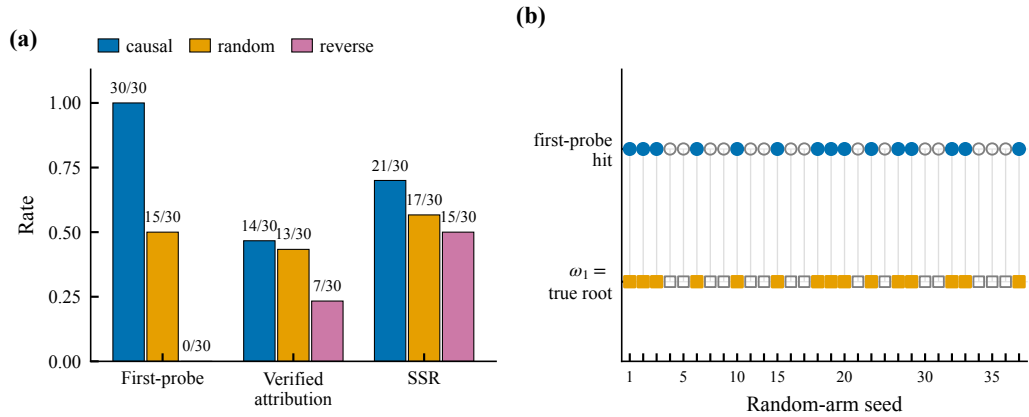


Figure 3: Exp-A commitment-order ablation and alignment audit (measured; snapshot mode; $n=30/\text{order}$; plant `me_force_zero_sea`). (a) First-probe hit, verified attribution, and SSR@ K by probe order. (b) Seed-level audit of the random arm: the executed first probe equals the committed ω_1 on every board (probing is bound to the commitment), so naming hits are exactly the shuffles that place the guilty layer first (15/15) — the random arm’s naming performance is carried entirely by accidental alignment with the ranking. Freeze attempts that produced no failure blackboard (seeds 7, 12, 14, 24, 25, 27, 28; data-contract rejections) are outside the campaign’s $n=30$.

Multi-plant replication (Phase-4 grid; snapshot mode; $n=10/\text{order}$). Table 4 replicates the ablation on the two additional plants of Section 5.1 under the identical engine and protocol. The result is a *conditional* replication that revises the mechanism claim. On the code-layer plant (`pd_comment_out_balance`), causal attains 9/10 first-probe, significantly above random’s 2/10 (7:0 discordant pairs, $p = 0.016$), but reverse attains 10/10: the reversed skeleton probes the code layer first *by construction*, so for a code-layer fault reverse is itself the aligned

Table 4

Phase-4 multi-plant replication — **measured** (snapshot mode; $n=10/\text{order}/\text{plant}$; $K=3$; hybrid rank; DeepSeek deepseek-flash; loop-token accounting). First-probe hit / verified attr. / SSR@ K per order.

Plant (a^*)	ω	First-probe	Verified attr.	SSR@ K	Loop tokens
pd_comment (python)	causal	9/10	5/10	5/10	0.85×10^5
	random	2/10	2/10	3/10	0.68×10^5
	reverse	10/10	5/10	5/10	0.51×10^5
de_swap (data)	causal	0/10	2/10	6/10	1.01×10^5
	random	4/10	6/10	8/10	0.82×10^5
	reverse	0/10	4/10	6/10	1.02×10^5

order, and the registered “strictly above reverse” clause cannot and does not hold (0:1). On the data-layer plant (de_swap_demand_supply_source), the hybrid rank never names the data layer (its smell rules cover force-zero and relaxed-DEP surfaces, not contract-valid semantic source swaps): causal and reverse both sit at 0/10 first-probe and the clause is falsified there; the only hits (4/10) are random shuffles that happened to align. Across all three plants and all 40 random-arm cells the executed first probe again equals ω_1 on every board, so the hits remain exactly the shuffles that target a^* — the empirical variable is which layer the shuffle places first, not whether execution obeys the commitment. Verified attribution shows no significant order effect on either new plant (seed-level repairability and multi-round refutation dominate at $K=3$). The grid also uncovered a plant-specific forward-variance class: in 12 of 22 freeze attempts for the data plant the swapped parameter never entered the generated model and the run solved to the un-faulted optimum ($\hat{z} = z^*$ to machine precision), leaving no failure blackboard; these attempts are excluded and replaced with fresh seeds, and their 55% rate is itself a measured property of semantic data faults in LLM-generated pipelines. The cross-plant reading is therefore not “causal order wins” but *alignment wins when the evidence ranking can see the fault layer*: visible for the model-layer plant through the force-zero smell and for the code-layer plant through the relaxed-DEP surface (and structurally through reverse), invisible for the semantic data fault, where naming fails for every fixed order and only chance alignment or multi-round executed refutation recovers the root cause. Figure 4 condenses this conditional pattern.

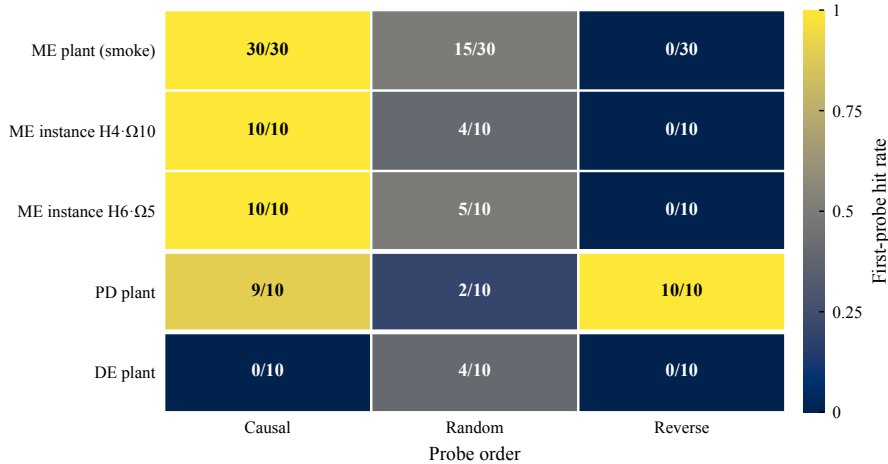


Figure 4: First-probe hit rate by plant/instance and probe order (measured; snapshot mode; $K=3$; DeepSeek deepseek-flash). The commitment-order effect is gated by fault visibility. Model-layer faults (smoke plant and both instance replications) are visible to the causal rank: causal = 1.0, random ≈ 0.4 –0.5, reverse = 0. The code-layer fault is visible through the relaxed-DEP surface from *both* ordered endpoints (causal 9/10, reverse 10/10 by construction) while random fails (2/10). The rank-invisible semantic data fault defeats every fixed order (causal = reverse = 0/10); only chance alignment (random 4/10) or multi-round executed refutation recovers it.

Table 5

Instance-family replication, model-layer plant — **measured** (snapshot mode; $n=10/\text{order}/\text{instance}$; $K=3$; hybrid rank; DeepSeek deepseek-flash; loop-token accounting).

Instance	ω	First-probe	Verified attr.	SSR@K	Loop tokens
H4_Omega10 ($T=4$)	causal	10/10	7/10	8/10	0.62×10^5
	random	4/10	2/10	6/10	0.94×10^5
	reverse	0/10	3/10	5/10	0.90×10^5
H6_Omega5 ($T=6$)	causal	10/10	5/10	8/10	0.67×10^5
	random	5/10	3/10	5/10	0.86×10^5
	reverse	0/10	1/10	6/10	0.69×10^5

Table 6

Exp-A preliminary (status-prior ω ; real $n=5$; last-comply attr.).

ω	First-probe hit	Attr. hit	SSR@K	Mean tokens
causal	4/5 (0.80)	0/5	1/5	1.38×10^5
random	3/5 (0.60)	0/5	0/5	1.38×10^5
reverse	0/5 (0.00)	1/5	1/5	1.40×10^5

Instance-family replication (snapshot mode; $n=10/\text{order}/\text{instance}$). Table 5 extends the model-layer plant to two further instances exported from the same generator with a longer horizon, a larger scenario set, and fresh base-data seeds (fam_H4_Omega10 and fam_H6_Omega5; each passed a no-plant end-to-end gate at relative gap 0.0% before faulting). The first-probe clause replicates at full strength: causal attains 10/10 on both instances and reverse 0/10; pooled over instances, causal versus reverse is 20:0 discordant pairs ($p = 2 \times 10^{-6}$) and causal versus random 11:0 ($p = 0.001$), with the executed first probe again equal to ω_1 on every board. The clause the $n=20$ grid could not resolve is resolved on the family and, after densification to $n=30$, on the main grid as well (causal versus reverse 8:1 discordant pairs, $p = 0.039$): verified attribution is strictly higher under causal (0.70 and 0.50) than under reverse (0.30 and 0.10) and random (0.20 and 0.30); pooled discordant pairs give causal versus reverse 8:0 ($p = 0.008$) and causal versus random 8:1 ($p = 0.039$). The smoke instance’s verified near-tie between causal and random (14/30 versus 13/30) does not recur, supporting the mechanism reading that alignment raises the probability of early verified attribution while multi-round refutation remains a second, slower route—consistent with the random arm being the most expensive in loop tokens on both instances while causal is the cheapest. Network size (nodes/arcs) is unchanged across the family, so this replicates along the horizon and scenario-count axes; the plant axis remains conditionally replicated as above.

Preliminary results (status-prior ω ; real $n=5$). Table 6 records the earlier status-prior run (last-comply attribution). The first-probe ordering is already directional (causal 4/5, random 3/5, reverse 0/5), but attribution collapses across all orders (at most 1/5): probing a layer and observing its backward response does not by itself establish guilt, including for the order that named the guilty layer first most often. Two limits of that apparatus follow. First, last-comply credit is not an attribution mechanism—the single reverse-order hit is last-comply credit, not a verified identification. Second, the status prior reads the failure surface, and surfaces mark symptom loci: on crash surfaces it points at the code layer regardless of where the defect was planted. Both limits motivated the evidence-informed redesign; the table is superseded as the primary Exp-A result by Table 3.

5.6. Exp-B: Baselines (internal and external)

Question. Relative to alternative diagnosis protocols that share the same blackboard, repair maps, and budget K , does evidence-informed causal Stackelberg achieve verified attribution that is at least competitive with a strong single-judge LLM, while retaining an auditable commit–probe–refute trail? Do stronger multi-call protocols (Debate, Reflexion) erase that advantage on accuracy, cost, or both?

Design. We compare causal Stackelberg (ω from evidence-informed ranking aligned to the causal skeleton) against the methods in Section 5.2: *internal* baselines (adversarial, sequential) and *external* verification baselines (Debate, Reflexion). Internal protocols stress fairness inside the same stack; external protocols stress the strong-LLM ceiling. Implementation may stage internal cells before external ones for cost control; scientifically they answer one comparison question and are reported in one table.

Expected conclusion. Verified attribution: Stackelberg $\geq$ adversarial $\gg$ sequential. SSR may favour sequential (blind repair). Debate may match Stackelberg on verified attribution but at higher tokens and lower audit coverage; Reflexion should not dominate both accuracy and cost. If adversarial or Debate strictly dominates Stackelberg on verified attribution at similar or lower cost with no audit gap worth claiming, the ranking/commitment design (or the paper claim) is revised before diagnostics scaling.

Measured results (evidence-informed; snapshot mode; internal only; $n=30$). Table 7 reports the expanded internal comparison on the same plant, engine, and K as Exp-A, using the Exp-A causal cells as the Stackelberg arm (paired, shared snapshots). Verified attribution: Stackelberg 14/30 versus adversarial 10/30 (6:2 discordant pairs, $p = 0.29$; the registered $\geq$ clause holds directionally) and sequential 0/30 (14:0 discordant pairs, $p = 1 \times 10^{-4}$; the $\gg$ clause holds). The cost dimension is decisive: adversarial’s verified rate sits below Stackelberg’s only non-significantly, yet it spends $2.1\times$ the loop tokens (1.36×10^5 versus 0.64×10^5), and its single-judge first-probe is 19/30 against Stackelberg’s committed 30/30.

The external rows extend the comparison on the identical thirty frozen blackboards. Debate (three lens-diverse voters, single round, mechanical majority with status-prior tie-break; no LLM-reported confidence) attains first-probe 19/30 — *cell-for-cell identical* to the single-judge baseline (0:0 discordant pairs) — and verified attribution 7/30, now significantly below Stackelberg’s (8:1 discordant pairs, $p = 0.039$), at $4.0\times$ Stackelberg’s loop tokens. Reflexion (single-judge accusation critiqued against episode memory) also attains first-probe 19/30 (cell-for-cell identical again), with verified attribution 15/30 (0.50, Wilson 95% CI [0.33, 0.67]): a point estimate above Stackelberg’s 0.47 (95% CI [0.30, 0.64]), where the two intervals overlap over most of their range and the paired difference is not significant (3:4 discordant pairs, $p = 1.0$), bought at $2.9\times$ the loop tokens without a committed order or its audit structure. Against Stackelberg, both external protocols lose first-probe naming decisively (11:0 discordant pairs each, $p = 0.001$). The registered revision clause — an external protocol strictly dominating Stackelberg on verified attribution at similar or lower cost — is therefore not triggered: on the tested engine, multi-calling the diagnoser neither improves naming over a single judge nor delivers verified attribution at competitive cost, although Reflexion’s non-significant verified edge is reported as an honest caveat against over-reading the Stackelberg advantage. Two scope conditions qualify the no-gain reading. First, all five arms share one engine (deepseek-flash, thinking disabled): whether multi-calling a diagnoser helps is the kind of question whose answer grows with judge capability, so the no-gain finding is claimed for this engine class, not for stronger judges; a full cross-engine campaign is registered future work. A registered spot-check on a second engine (qwen3.8-flash, five shared blackboards $\times$ three order arms, resuming the identical frozen boards) replicates the commitment-side structure with zero exceptions — causal first-probe 5/5, random 3/5 with every hit again exactly the aligned shuffle, reverse 0/5 — while verified attribution is uniformly rarer there ($\leq 1/5$ per order), consistent with the funnel decomposition: naming is commitment-side and engine-robust, round-one confirmation rides on the repair path and is engine-sensitive; the cells are never pooled across engines. Second, the common budget $K=3$ is the least favourable to Reflexion’s memory-based revision, so a *budget-tilted challenger sweep* re-ran Reflexion alone with $K=5$ on all thirty boards: verified attribution is unchanged at 15/30 (9 at $K=1$, 15 by $K=2$, flat through $K=5$; paired against Stackelberg’s $K=3$ cells, 3:2 discordant pairs, $p = 1.0$) while mean loop tokens rise to $3.1\times$ Stackelberg’s — budget depth is not what limits Reflexion here, and the revision clause stays untriggered. Figure 5 shows the accuracy–cost trade-off.

Preliminary results (status-prior; real $n=5$, internal only). Table 8: last-comply attribution was 0/5 for all three methods, while SSR diverged (sequential repaired 2/5 blind, stackelberg 1/5, adversarial 0/5). The attribution failure is uniform across protocol families—single-judge accusation, committed causal probing, blind reverse-chain repair—so no protocol’s comply/deflect responses identified the planted layer under last-comply accounting; conversely, sequential’s blind-repair SSR shows that eventual repair is likewise uninformative about root cause. These two observations motivated the verified attribution accounting and the evidence-informed rank of the redesign. External baselines were not run in that preliminary pass.

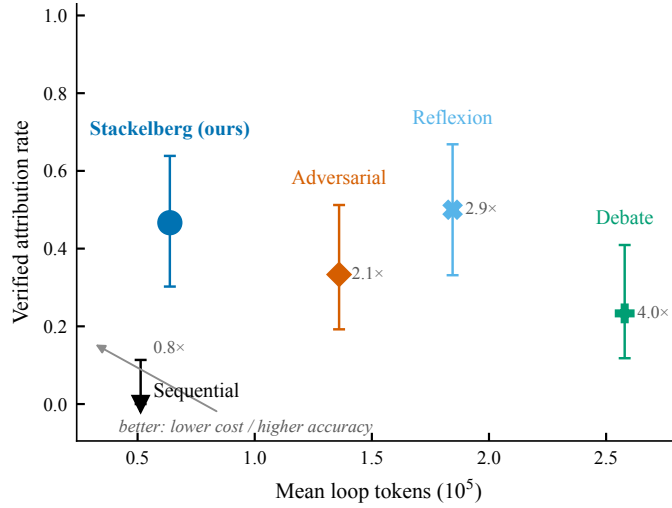


Figure 5: Verified attribution versus mean loop-token cost across the five Exp-B methods ($n=30$ shared frozen blackboards each; error bars: Wilson 95% CIs). Stackelberg sits on the efficient frontier at the lowest token cost; adversarial trails its point estimate only non-significantly at 2.1 $\times$ the cost, and the external multi-call protocols cost 2.9 $\times$ (Reflexion) to 4.0 $\times$ (Debate). Reflexion’s higher point estimate (0.50 vs 0.47) is not significant — its interval overlaps Stackelberg’s over most of its range (3:4 discordant pairs, $p = 1.0$).

Table 7

Exp-B unified baselines — **measured** (evidence-informed; snapshot mode; $n=30$; $K=3$; shared frozen blackboards across all methods; loop-token accounting). Primary: verified attribution.

Family	Method	Verified attr.	First-probe	SSR@K	Mean loop tokens
Proposed	stackelberg (causal)	14/30 (0.47)	30/30 (1.00)	21/30	0.64×10^5
Internal	adversarial	10/30 (0.33)	19/30 (0.63)	13/30	1.36×10^5
Internal	sequential	0/30 (0.00)	0/30 (0.00)	10/30	0.51×10^5
External	debate (3-voter panel)	7/30 (0.23)	19/30 (0.63)	7/30	2.58×10^5
External	reflexion (memory critique)	15/30 (0.50)	19/30 (0.63)	16/30	1.84×10^5

Table 8

Exp-B preliminary internal baselines (status-prior; real $n=5$).

Method	Attr. hit	First-probe hit	SSR@K	Mean tokens
stackelberg (causal)	0/5 (0.00)	4/5 (0.80)	1/5	1.38×10^5
adversarial	0/5 (0.00)	2/5 (0.40)	0/5	2.35×10^5
sequential	0/5 (0.00)	0/4 (0.00)	2/5	0.86×10^5

5.7. Exp-C: Inspection diagnostics

Question. Is commitment operational in the implementation—not merely additional LLM calls? As the budget K grows, does verified attribution rise under causal commitment, and are inconsistent defects overturned by executed refutation?

Design. We sweep K for causal Stackelberg (and, as references, adversarial and sequential) on the same injection grid, recording verified attribution versus K , defect rates, and the fraction of defects/compliances overturned by re-solve. Analysis-mode payoffs (u_L, u_F) from Section 4.4.2 may be reported as secondary trajectory summaries.

Table 9

Exp-C diagnostics — **measured** (offline re-analysis of the completed smoke-grid episodes; $n=30$ shared blackboards per method; verified attr. cumulative within the completed $K=3$ episodes).

Method	$K=1$	$K=2$	$K=3$	Probes/cell	Deflect	Comply overturn
stackelberg causal	14/30	14/30	14/30	1.8	26%	49%
stackelberg random	9/30	12/30	13/30	2.1	32%	43%
stackelberg reverse	0/30	7/30	7/30	2.2	22%	71%
adversarial	8/30	10/30	10/30	2.3	16%	60%
sequential	0/30	0/30	0/30	2.4	7%	56%
debate	7/30	7/30	7/30	2.5	56%	64%
reflexion	9/30	15/30	15/30	2.2	14%	55%

Expected conclusion. Verified attribution rises with K faster under causal Stackelberg than under random/reverse or non-committed baselines; deflects that contradict re-solve are overturned at high rate. Flat curves in K or near-zero overturn rates indicate that commitment is not operational.

Measured results (offline re-analysis of the completed grid). Table 9 reports an offline reconstruction of per-probe dynamics from the event logs of the smoke-grid campaigns above (seven method arms, all on the same thirty frozen blackboards; re-analysis adds no runs): verified-at-round- k is read cumulatively within the completed $K=3$ episodes, a comply is *overturned* when the executed re-solve re-enters diagnosis, and a deflect is *false* when it comes from the eventually verified layer. Three readings follow. First, the curves are step functions positioned by where the guilty layer sits in the order: under causal commitment all fourteen verified attributions occur at the *first* probe (14/30 at $K=1$, saturated — the first-probe confirmations of Section 5.3), reverse reaches zero at $K=1$ and its full 7/30 only at $K\geq 2$, and random lies between (9 hits at $K=1$, 12 by $K=2$, 13 at $K=3$). The registered “rises faster under causal” clause thus holds in the step-function sense, and the deeper diagnostic is that every method *except* random is flat from $K=2$ to $K=3$ (random’s single $K=3$ recovery is one more shuffle whose order reaches the guilty layer only on the third probe): within this budget the operative variable is the alignment of ω_1 with a^* , not budget depth (for the non-Reflexion methods a $K>3$ sweep was not run; Reflexion was swept to $K=5$ as the budget-tilted challenger of Section 5.6 and gains nothing beyond round two). Second, deflects are near-truthful with one qualified exception: across all 465 recorded probes by the seven methods — the complete event-log population (208 of the 210 episodes contribute probes; the remaining two crashed at the execution stage before any probe) — 116 deflects were recorded, 113 of them from innocent downstream layers, and within the three committed order arms the guilty layer complied at first contact on every board. The exception is instructive: the only three guilty-layer deflections (two Debate cells, one Reflexion cell) all occur *after* the guilty layer’s round-one complying repair was overturned by the re-solve — late-stage re-accusation defences arguing that a passing solver status leaves no error to own, not concealment of visible guilt. This is still the truthful-best-response behaviour the commitment-plus-refutation mechanism is designed to induce where it binds (single-plant caveat: the model-layer plant’s guilt is visible in its own output). Third, executed refutation is load-bearing: 43–71% of complies are overturned by the re-solve (the repair claimed success but the objective did not recover), so a comply is never taken on trust. Debate’s 56% deflect rate is the panel’s data-layer accusations being rejected by the data layer’s own backward check; Reflexion’s $9 \rightarrow 15$ jump between $K=1$ and $K=2$ reflects its memory-driven revision escaping bad first attributions, consistent with its multi-round profile in the external comparison. Figure 6 plots the step curves.

5.8. Failure-mode analysis

Table-level rates do not reveal where the mechanism breaks, so we additionally inspected the per-node event logs and repair artifacts of every measured cell above and of the intervening single-run smokes. Three recurrent failure families emerged; each is a property of the implemented pipeline under layered faults, and each motivated a specific element of the measured design or of the pending expanded grid.

Traceback-as-oracle. On crash surfaces the failure evidence points at the generated code, and ranking that trusts the surface tips the code layer. Under an intermediate evidence-informed variant with LLM-only ranking (same grid, $n=3$), two of three causal cells committed a code-first ω , spent the budget on code- and data-layer overturns, and never

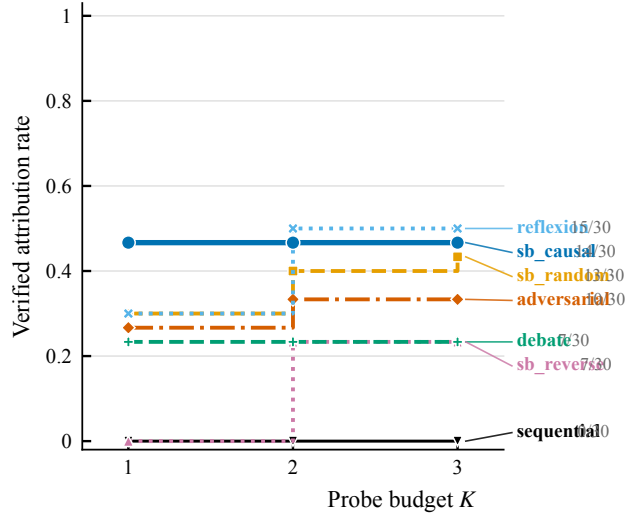


Figure 6: Verified attribution as a function of probe budget K (Exp-C offline re-analysis; $n=30$ shared blackboards per method; cumulative within the completed $K=3$ episodes). The curves are step functions positioned by the rank of the guilty layer in the committed order ω : causal commitment saturates at $K=1$ (14/30), reverse reaches its plateau only at $K \geq 2$, sequential never leaves zero, and Reflexion’s $9 \rightarrow 15$ jump between $K=1$ and $K=2$ reflects memory-driven revision of bad first attributions. Every method except random is flat from $K=2$ to $K=3$.

reached the model layer; verified attribution was 0/9 over that grid. This is the practical form of the symptom-locus ambiguity of Section 4.4.1: the traceback marks where the pipeline broke, not where the defect was planted. The hybrid ranking of the measured tables counters it with an explicit symptom-signature rule—a forced-zero balance is model-layer evidence—while still ignoring LLM-reported confidence.

Stale evidence at repair time. A correct tip does not guarantee a verified attribution: in that same intermediate variant, the one cell that tipped the model layer still failed, because its backward step consumed the pre-fault forward output, in which the planted constraint was absent; the repair rewrote an unrelated model element and was overturned by the re-solve. The measured implementation presents the repairing agent with the current (post-fault) output and restricts the repair to a minimal strip of the implicated constraint rather than a free regeneration.

Residual repair-path variance. The measured campaigns retain a regeneration instability downstream of a correct model-layer repair: a code-layer regeneration can fail outright (in a single-run smoke it raised a key error and left a large objective gap despite a correct strip); one sequential grid cell terminated before any model call; and replicate variance remains high—the adversarial arm reached a verified attribution in a single-run smoke but 0/3 on the grid. Success can also occur without attribution: seven causal cells reached *Practical Optimal* through a downstream repair while their attributed layer settled on the code layer (the stolen-attribution branch of the Exp-A confirmation funnel). These residuals concentrate in the paths the deterministic data-layer contract now constrains; the no-plant gate and the re-established tables of Section 5.4 bound them before the expanded grid is interpreted, and the overturn and deflect diagnostics of Exp. 5.7 will quantify them at scale.

Rank-invisible semantic faults. The Phase-4 data-layer plant exposes a fourth family. A contract-valid semantic source swap leaves every structural signal intact—the validator passes, the model formulates, the code runs—and the failure surface (OPTIMAL with a gap) is indistinguishable from a modeling fault, so the hybrid rank named the data layer first in 0 of 10 boards and causal naming failed across the board (Table 4). Worse, in 12 of 22 freeze attempts the swapped parameter never entered the generated model at all and the run solved to the un-faulted optimum to machine precision—a silent fault. Semantic data faults therefore require evidence beyond the current surface smells (e.g., provenance checks on which catalog table feeds which parameter), a design revision registered for future work rather than patched post hoc.

Claim boundary. Comparisons are scoped to layered-error natural-language optimization on the TSLP ECR pipeline. We do not claim gains for general LLM reasoning, nor that inspection is the unique protocol that can eventually repair a failed solve. The intended positive claim is that evidence-informed causal-order committed inspection matches (or exceeds) strong single-judge naming accuracy while improving verifiable attribution and auditability relative to unordered commitment and to non-committed verification baselines under the injection regime above. The visibility reading carries one further threat: each fault layer is represented by a *single* plant, so “the model layer is visible to the rank / the semantic data layer is not” is established for the tested plant families, not for layers as such — a different model-layer plant without a symptom signature could be as rank-invisible as the semantic swap. Within-layer plant replication is therefore registered alongside the data-layer evidence revision before the visibility claim is read as a layer property.

6. Conclusion

We recast diagnosis–repair in an LLM optimization pipeline as a Stackelberg inspection game and asked what the inspector’s commitment is actually worth. The empirical answer, across three fault plants, three problem instances, five diagnosis protocols, and the 465 probe rounds recorded across the seven arms of the shared grid, is precise about the mechanism: *committed alignment carries the information*. Probing is bound to the committed order — the executed first probe equals ω_1 on every board ever run — so first-probe naming is exactly the ranking’s alignment, and the ranking sees the model layer (through symptom signatures) and the code layer (through relaxed-solve surfaces) but not contract-valid semantic data faults, for which naming failed for every fixed order. Naming is not confirmation: on the main grid only 14 of the 30 aligned first probes are settled by the round-one re-solve (9 overturned repairs, 7 attributions stolen by downstream regeneration), while on the reverse arm 8 misaligned probes absorbed the upstream fault outright and succeeded without verified attribution. The order-effect claim is therefore conditionally replicated, not universal, and we register the corresponding design revision (provenance-style evidence for the data layer) rather than patching it post hoc.

The quantitative picture is consistent with that mechanism. On the shared frozen blackboards of the main grid, causal Stackelberg names the guilty layer with its first probe on every board and dominates single-judge, debate, and reflexion naming (11:0 discordant pairs each); verified attribution reaches 0.60 versus 0.20 under reverse commitment on the instance family (pooled paired $p = 0.008$) and 0.47 versus 0.23 on the densified main grid (8:1, $p = 0.039$); and the budget diagnostics show that nothing improves after the second round within the budgets tested ($K=3$ for all arms; $K=5$ for Reflexion as the tilted challenger) — the guilty layer’s position, not budget depth, is what the curves respond to. On the single tested engine, external multi-call protocols do not overturn the design: debate and reflexion name no better than one judge, debate’s verified attribution is significantly lower (8:1, $p = 0.039$) at 4.0× the loop tokens, and reflexion’s non-significant verified edge (0.50 versus 0.47) costs 2.9× the loop tokens without a committed order or its audit trail. Two mechanism checks close the loop between theory and logs: under commitment the guilty layer complied at first contact on every committed-arm board (the three deflections observed, all in external arms, follow an overturned repair), while 43–71% of complying repairs were overturned by the re-solve — verification, not cheap talk, is what carries attribution.

The claims are bounded accordingly: the first-contact-compliance result is established on the model-layer plant (whose guilt is visible in its own output; the three guilty-layer deflections observed, all in external arms, follow an overturned repair) and is silent on the semantic data plant, whose 55% silent-fault rate means most injected faults never produced a failure board and deflect behaviour was never exercised; the instance family varies horizon and scenario count at fixed network size, the main campaigns run a single engine under a frozen implementation (the second-engine spot-check covers five boards), K is capped at three (five for the Reflexion challenger), and each fault layer is represented by one plant, so visibility is a property of the tested plant families rather than of layers as such. Within those bounds the contribution stands as a mechanism, a proposition with matching dynamics, and a measured protocol: commitment buys naming accuracy and cheaper verifiable attribution precisely when the committed ranking can see the fault — and the logs show why. Extending visibility to the semantic data layer, replicating within layers across additional plants, scaling the network, and a full cross-engine campaign (a second-engine spot-check already replicates the commitment-side alignment structure with zero exceptions) are the registered next steps.

Data and code availability

The frozen instance data, fault-injection plants, run manifests, and per-node event logs supporting every measured table, together with the pipeline implementation, will be released upon acceptance.

References

- Q. Wu et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *COLM*, 2024.
- Y. Xiao et al. Chain-of-experts: When LLMs meet complex operations research problems. In *ICLR*, 2024. <https://openreview.net/forum?id=HobyL1B9CZ>.
- A. AhmadiTeshnizi, W. Gao, M. Udell. OptiMUS: Scalable optimization modeling with (MI)LP solvers and large language models. In *ICML*, PMLR 235:577–596, 2024. <https://arxiv.org/abs/2402.10172>.
- A. AhmadiTeshnizi, W. Gao, H. Brunborg, S. Taleai, M. Udell. OptiMUS-0.3: Using large language models to model and solve optimization problems at scale. *arXiv:2407.19633*, 2024.
- B. Yang, Q. Zhou, J. Li, C. Su, P. Angeloudis, S. Hu. ORThought: Benchmarking and automating logistics optimization modeling via structured LLM reasoning. *arXiv:2508.14410*, 2025.
- Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, I. Mordatch. Improving factuality and reasoning in language models through multiagent debate. *arXiv:2305.14325*, 2023.
- T. Liang, Z. He, W. Jiao, et al. Encouraging divergent thinking in large language models through multi-agent debate. In *EMNLP*, pages 17889–17904, 2024.
- N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.
- S. Zhang, M. Yin, J. Zhang, et al. Which agent causes task failures and when? On automated failure attribution of LLM multi-agent systems. In *ICML*, PMLR 267:76583–76599, 2025. <https://arxiv.org/abs/2505.00212>.
- M. Chen, J. Wang, F. Mu, et al. Seeing the whole elephant: A benchmark for failure attribution in LLM-based multi-agent systems. In *ACL*, 2026. <https://arxiv.org/abs/2604.22708>.
- Y. Wang, W. Wu, J. Wang, Q. Wang. From flat logs to causal graphs: Hierarchical failure attribution for LLM-based multi-agent systems. *arXiv:2602.23701*, 2026.
- J. Shah. Causal Agent Replay: Counterfactual attribution for LLM-agent failures. *arXiv:2606.08275*, 2026.
- X. Yi, Z. Zhou, C. Cao, Q. Niu, et al. From debate to equilibrium: Belief-driven multi-agent LLM reasoning via Bayesian Nash equilibrium. In *ICML*, 2025. <https://proceedings.mlr.press/v267/yi25c.html>.
- B. Wang. A Stackelberg framework for resource-aware LLM agents: Learning, repair, and conditional guarantees. *arXiv:2606.23026*, 2026.
- H. von Stackelberg. *The Theory of the Market Economy*. Oxford University Press, 1952.
- R. Avenhaus, B. von Stengel, S. Zamir. Inspection games. In R. J. Aumann and S. Hart (eds.), *Handbook of Game Theory with Economic Applications*, vol. 3, ch. 51, pp. 1947–1987. Elsevier, 2002.
- M. Maschler. A price leadership method for solving the inspector’s non-constant-sum game. *Naval Research Logistics Quarterly*, 13(1):11–33, 1966.
- D. Korzhuk, Z. Yin, C. Kiekintveld, V. Conitzer, M. Tambe. Stackelberg vs. Nash in security games: An extended investigation of interchangeability, equivalence, and uniqueness. *Journal of Artificial Intelligence Research*, 41:297–327, 2011. <https://doi.org/10.1613/jair.3269>.
- M. Tambe. *Security and Game Theory: Algorithms, Deployed Systems, Lessons Learned*. Cambridge University Press, 2011.
- R. Poiani, C. Stirbu, A. M. Metelli, M. Restelli. Optimizing empty container repositioning and fleet deployment via configurable Semi-POMDPs. *IEEE Transactions on Intelligent Transportation Systems*, 25(5):4704–4711, 2024. <https://doi.org/10.1109/TITS.2023.3329677>.
- W. Wang, C. Diao, W. He, Z. Jin, Z. Yang. Collaborative optimization of container liner slot allocation and empty container repositioning within port clusters. *Journal of Marine Science and Engineering*, 13(1):159, 2025. <https://doi.org/10.3390/jmse13010159>.
- J. Cai, Y. Huang, C. Diao, Z. Jin. Joint optimization of multi-period empty container repositioning and inventory control based on adaptive particle swarm algorithm. *Journal of Marine Science and Engineering*, 13(6):1113, 2025. <https://doi.org/10.3390/jmse13061113>.
- T. Liu, J. Yang, Y. Yin. Toward LLM-agent-based modeling of transportation systems: A conceptual framework. *Artificial Intelligence for Transportation*, 1:100001, 2025. <https://doi.org/10.1016/j.ait.2025.100001>.